데이터 이행 도구 — DataFlex.PG

> 2026-10-26 초안 (도구 소개 문서 — 제안서가 아니므로 일정 · 인력 · 문의처 같은 제안 관련 항목은 두지 않는다).
> DataFlex.NET 소개 문서를 바탕으로 PostgreSQL 판의 사실로 바꿨다.

1. 개요

DataFlex.PG 는 PostgreSQL 데이터 이행의 **매핑 정의 · 이행 스크립트 생성 · 실행 · 결과 확인**을 하나의 리포지토리 DB 위에서 처리하는 도구입니다. 매핑은 화면에서 정의해 DB 에 보관하고, 실제 데이터 이행은 도구가 생성한 PL/pgSQL 스크립트가 DB 안에서 실행되면서 데이터를 적재합니다.

구분	내용
대상	PostgreSQL → PostgreSQL 데이터 이행 (가공 · 변환 포함)
형태	Windows 클라이언트 + PostgreSQL 리포지토리 DB (dataflex 롤 · 스키마 dataflex · dataflex_dup · dataflex_back)
결과물	매핑 정의, PL/pgSQL 이행 스크립트, 실행 순서, 실행 기록
특징	매핑 한 곳에서 스크립트 · 현황 · 기록이 모두 나온다

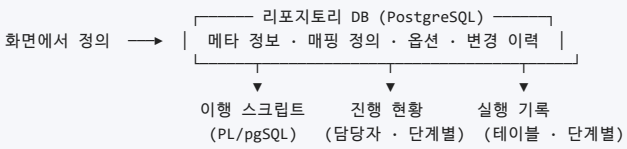
2. 배경 — 이행 프로젝트의 반복되는 위험

기존 방식은 매핑 정의서(엑셀), 수작업으로 작성한 이행 SQL, 로그 파일, 대사용 엑셀이 각각 따로 관리됩니다. 대상 테이블이 수백 개 규모가 되면 다음 세 가지 위험이 반복해서 나타납니다.

위험	내용	드러나는 시점
정의와 SQL 의 불일치	컬럼 하나가 바뀌면 정의서 · 이행 SQL · 검증 쿼리를 각각 고쳐야 하며, 그중 하나가 누락됨	리허설 · 본 이행 당일
작성자에 따른 품질 편차	병렬 처리, 인덱스 분리 · 복구 여부가 작성자마다 다름. 분리한 인덱스를 원래 옵션대로 복구했는지 보장할 수 없음	성능 미달 · 이행 후 장애
진행 상황의 불투명	매핑 진척, 리허설 실패 테이블과 원인, 전체 소요 시간을 요청할 때마다 수작업으로 집계	일정 관리 · 보고

세 위험의 원인은 같습니다. **같은 정보가 여러 곳에 따로 관리된다는 것**입니다.

3. 해결 방안 — 매핑은 한 곳에서 관리, 나머지는 자동 생성

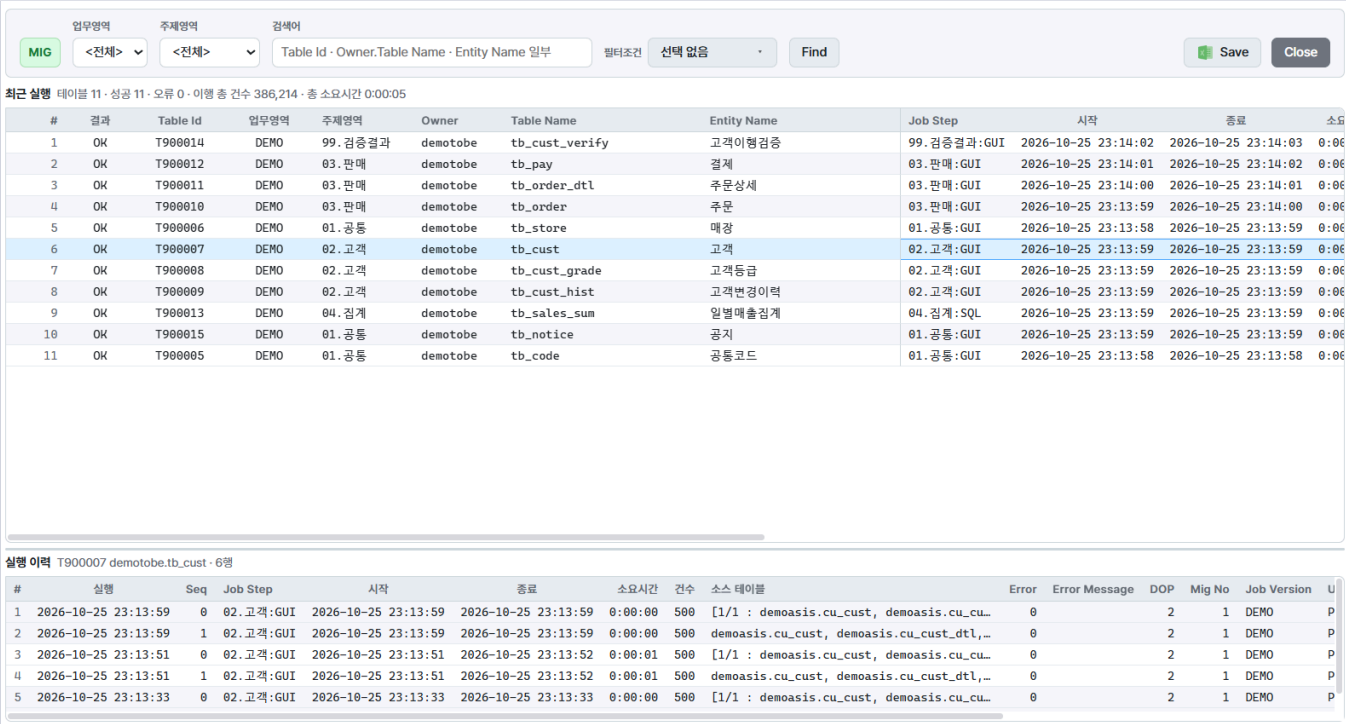


- 매핑을 고치면 스크립트를 **다시 생성**합니다. 수작업으로 맞출 대상이 없습니다.
- 스크립트는 누가 생성해도 **같은 규칙**으로 나옵니다. 적재 방식은 작성자의 습관이 아니라 테이블마다 지정한 옵션에서 결정됩니다.
- 실행 결과는 로그 파일이 아니라 **DB 테이블**에 기록됩니다. 현황은 집계하는 것이 아니라 조회하는 것이 됩니다.
- 데이터는 **DB 안에서** 이동합니다. 클라이언트 PC 나 중간 서버를 거치지 않으므로, 처리 속도는 DB 자원이 허용하는 만큼 확보되고 데이터가 DB 밖으로 나오지 않습니다.

4. 기대 효과

구분	기존 (엑셀 + 수작업 SQL)	DataFlex.PG 적용
매핑 정의	엑셀 파일, 담당자마다 최신본이 다름	리포지토리 DB 한 곳에서 관리, 변경 이력 자동 기록
구조 변경 대응	정의서 · SQL · 검증 쿼리를 각각 수정	차이를 비교해 선택 반영 → 스크립트 재생성
적재 SQL 품질	작성자의 습관에 의존	테이블별 옵션에서 동일한 틀로 생성
인덱스 · 제약 · 트리거	분리와 복구 모두 수작업	분리 → 적재 → **원래 옵션 그대로** 복구

구분	기존 (엑셀 + 수작업 SQL)	DataFlex.PG 적용
동시 작업	파일 덮어쓰기 위험	테이블 단위 잠금
실행 결과 확인	로그 파일을 개별 확인	리포지토리 DB 에 기록, 화면 조회 · 엑셀 저장
진행 현황 보고	요청 시마다 수작업 집계	담당자 · 주제영역 · 진행 단계별 상시 조회
리허설 반복	회차마다 준비 비용 발생	수정 → 재생성 → 재실행의 고리가 짧음

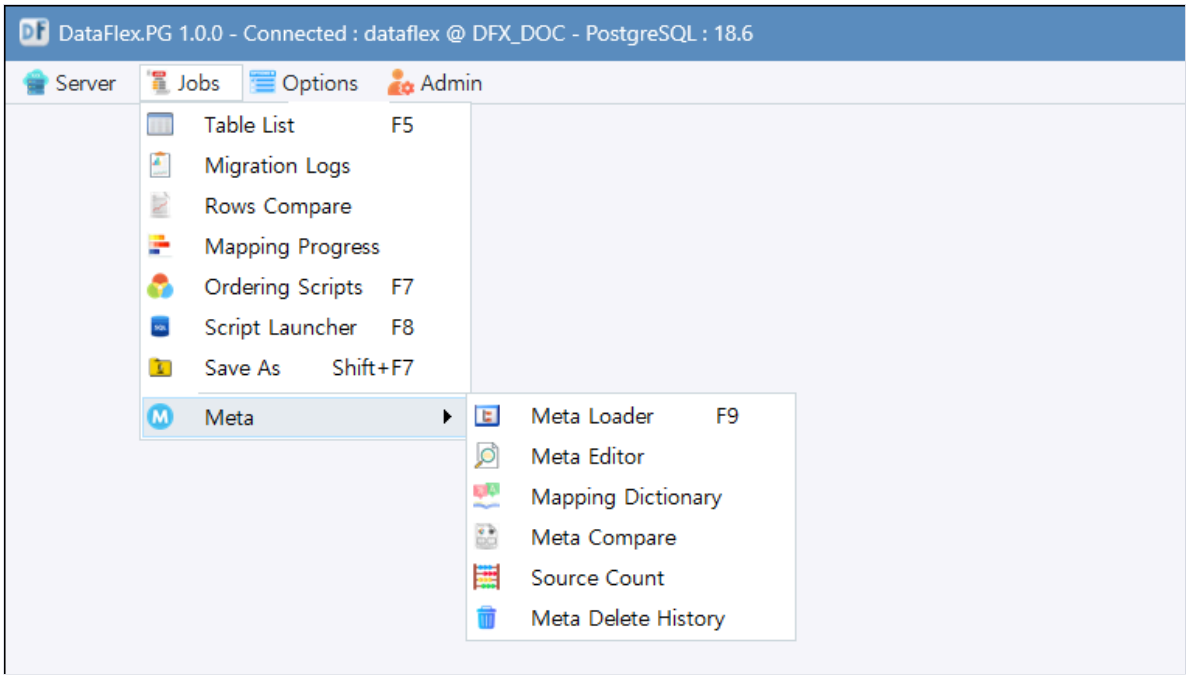


Migration Logs — 위는 최근 실행입니다. 이 프로젝트의 대상 테이블 11개가 각각 언제 시작해 몇 건을 몇 초에 옮겼는지가 한 줄씩 보입니다(성공 11 · 오류 0 · 386,214건 · 5초). 아래는 선택한 테이블(tb_cust)의 실행 이력으로, 리허설을 돌릴 때마다 회차가 쌓이며 소스 테이블 · 병렬도 · 오류 메시지 · 실행한 세션까지 남습니다. 로그 파일을 열어 보는 것이 아니라 조회하는 것이고, 오류만 · PL/pgSQL 방식만 · 선이행 대상만으로 거르거나 엑셀로 저장합니다.

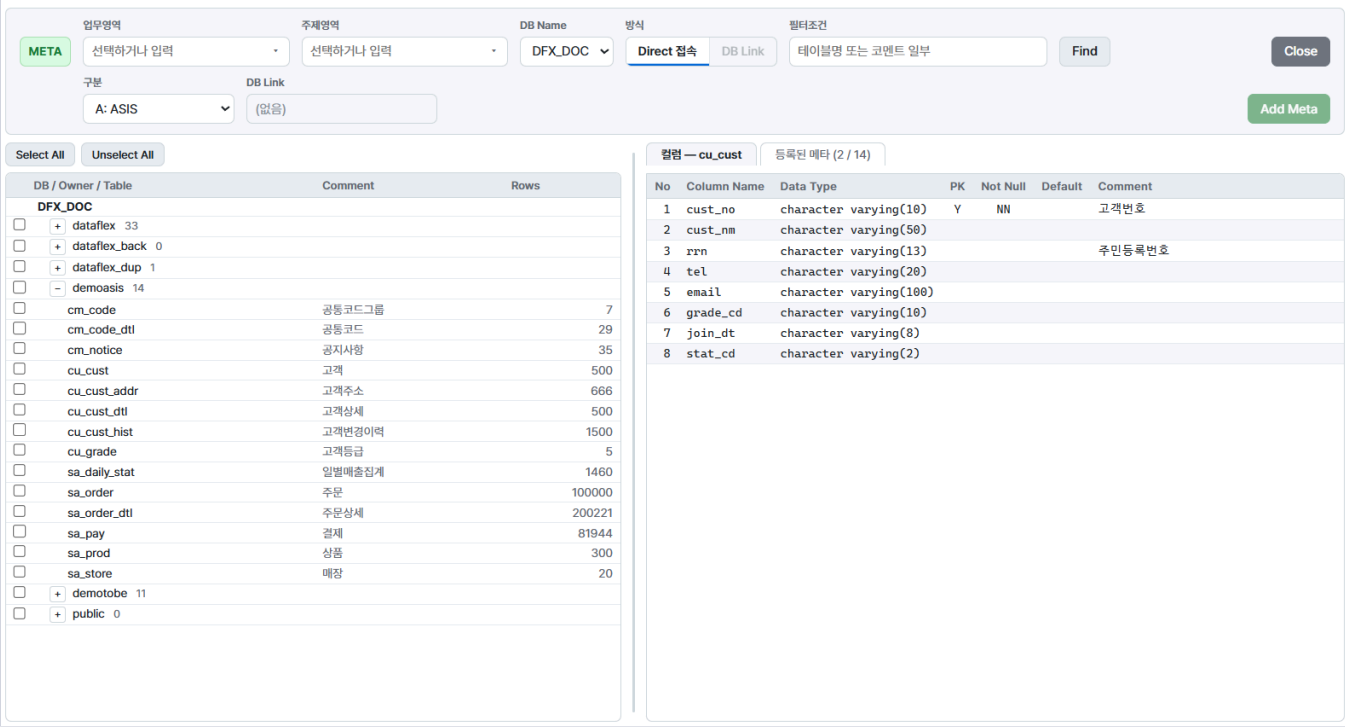
5. 기능 구성

5.1 업무 흐름

단계	화면	하는 일
① 등록	Meta Loader	원본 · 대상 DB 의 테이블 / 컬럼 정보를 리포지토리 DB 에 등록 (직접 접속 또는 DB Link — postgres_fdw 외부 서버)
② 매핑	Table Map · Column Map	대상 테이블별 소스 · 조인 · 적재 방식, 컬럼별 변환식 정의
③ 생성	Save As · Ordering Scripts	테이블별 스크립트와 주제영역별 루트 스크립트 생성, 실행 순서 지정
④ 실행	Script Launcher	루트 스크립트를 실행 노드에 배정해 실행, 타임라인으로 진행 확인
⑤ 확인	Migration Logs · Mapping Progress · Source Count · Rows Compare	테이블 · 단계별 건수 · 소요 시간 · 오류, 매핑 진척, 원본 건수, 건수 대사



Jobs 메뉴 — 업무 흐름의 화면이 한 메뉴에 차례로 있습니다: Table List · Migration Logs · Rows Compare · Mapping Progress · Ordering Scripts · Script Launcher · Save As, 그리고 Meta 아래에 Meta Loader · Meta Editor · Mapping Dictionary · Meta Compare · Source Count · Meta Delete History. 매핑 화면(Table Map · Column Map)은 메뉴가 아니라 Table List 의 테이블에서 엽니다.

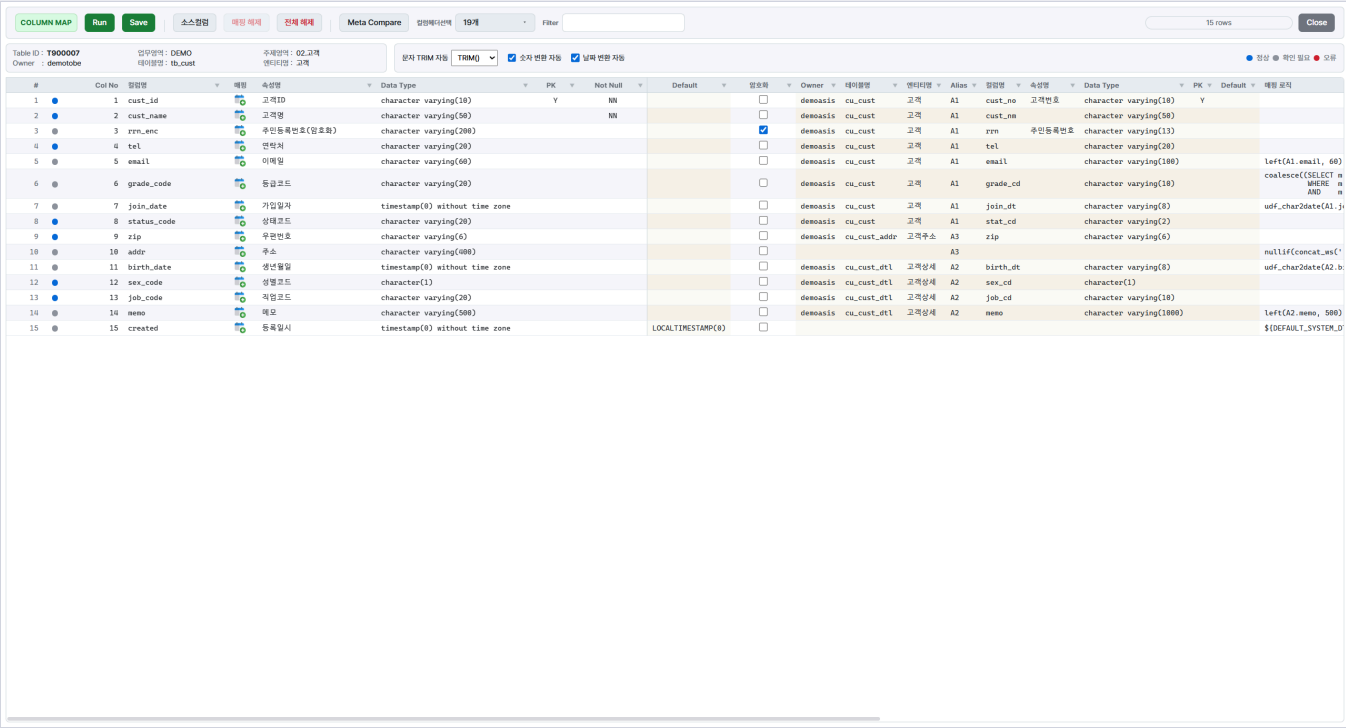


Meta Loader — 원본 · 대상 DB 의 테이블 정보를 리포지토리 DB 에 등록하는 첫 화면입니다. 접속 목록에서 DB 를 선택하면(DFX_DOC) 스키마 → 테이블 트리가 코멘트 · 건수와 함께 뜨고, 선택한 테이블의 컬럼(이름 · 타입 · PK · Not Null · Default · 코멘트)을 확인한 뒤 Add Meta 로 넣습니다. 원본(ASIS) · 대상(TOBE) 구분을 정해 등록하며, 사전을 읽는 방식은 그 DB 에 직접 붙는 것과, 접속 정보 없이 외부 서버(postgres_fdw)만 받은 운영 DB 를 위해 DB Link 를 타는 것 두 가지입니다. 이미 등록된 메타(14)는 오른쪽 탭에서 봅니다.

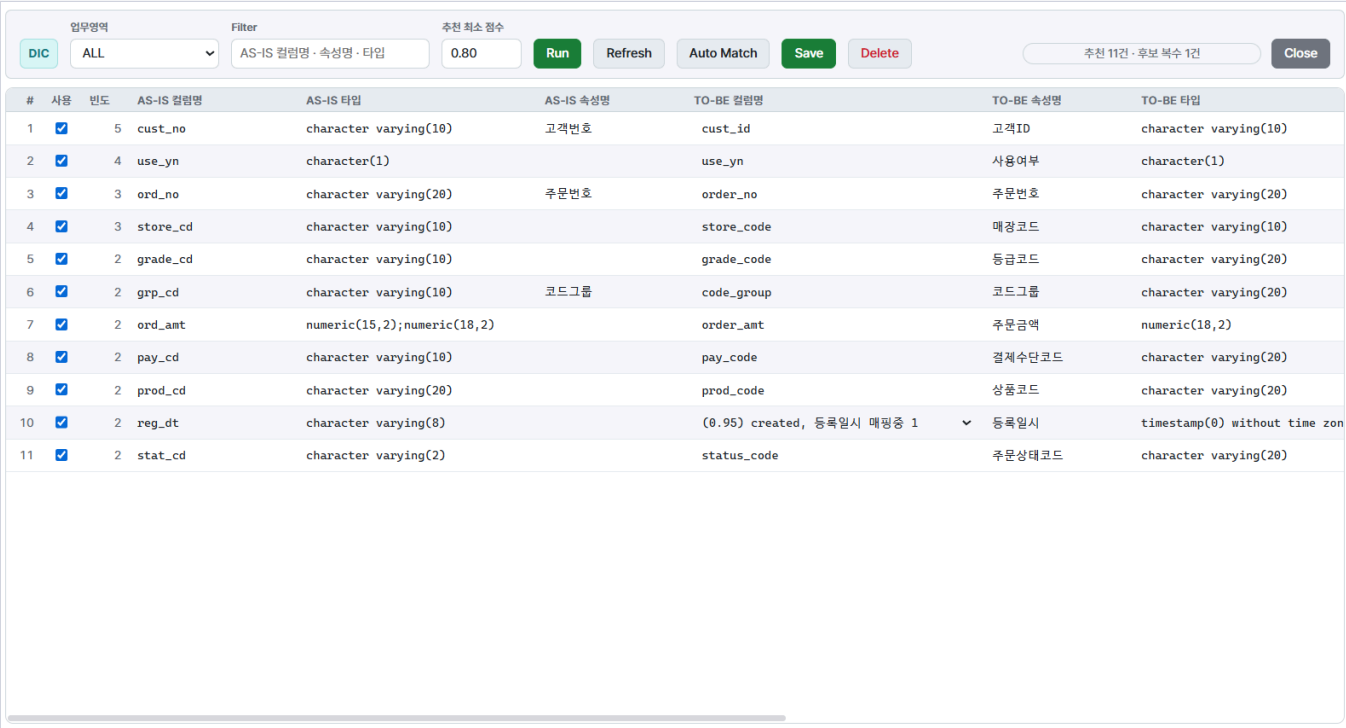
5.2 매핑

- **다중 소스** — 소스 테이블을 조인하거나 인라인 뷰로 묶어 하나의 대상 테이블에 적재. 한 대상 테이블에 소스 묶음을 여러 개 구성 가능
- **원본 위치** — 리포지토리 DB 안의 다른 스키마, 같은 서버의 다른 DB, 다른 서버 모두. 같은 DB 밖의 원본은 postgres_fdw 외부 서버로 읽으며, 외부 서버 · 사용자 매핑은 Connections 화면의 Set Up Foreign Server 로 만든다
- **컬럼 변환** — 변환식을 SQL 로 기술 (구문 강조 편집기). 코드 변환 테이블, 암호화 함수 적용, 프로젝트 공용 변수 지원
- **사전 경고** — 원본 길이가 대상보다 긴 컬럼을 매핑 단계에서 표시

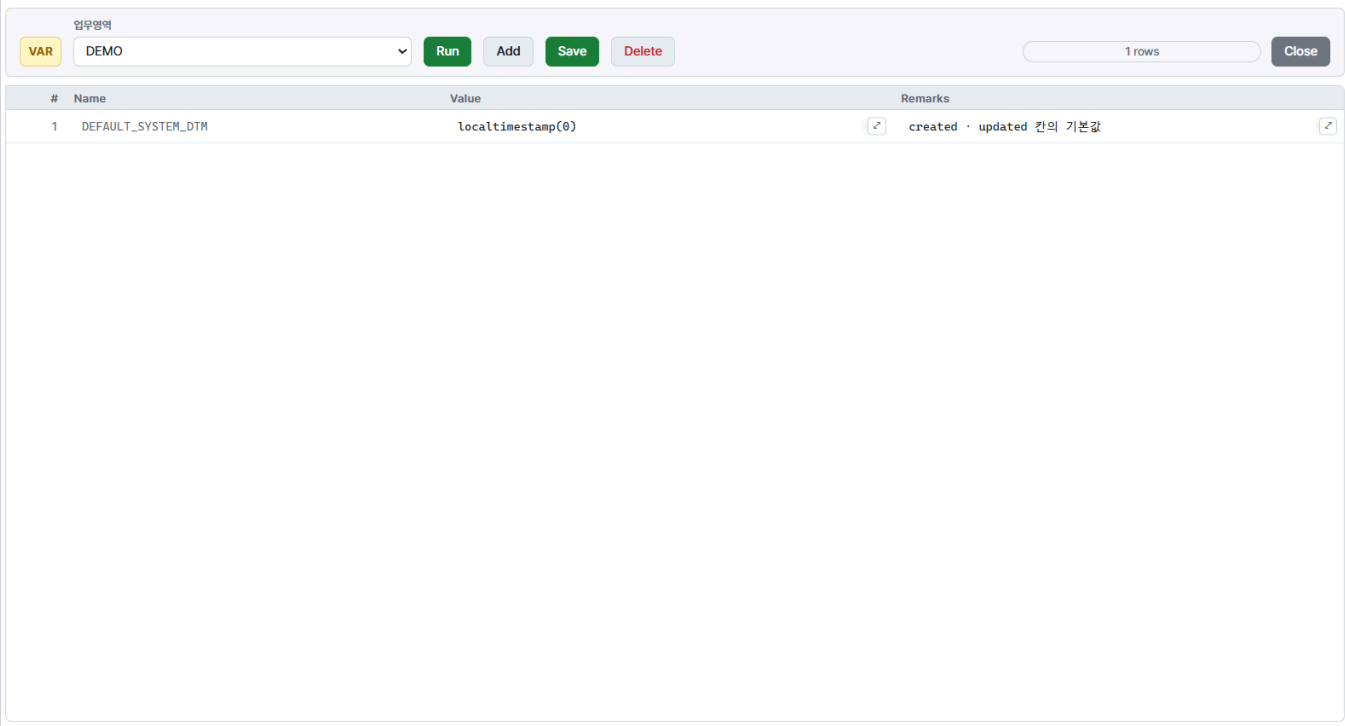
- **자동 매핑(Auto Map)** — 용어사전 → 매핑중(다른 테이블에서 이미 연결한 짝) → 유사도(약어 · 끝 분류어 · 데이터 타입) 순서로 소스 컬럼을 찾아 붙이고, 무엇을 근거로 붙였는지를 매핑 설명에 남김
- **매핑 사전(Mapping Dictionary)** — AS-IS 컬럼을 여러 테이블에 나오는 빈도순으로 나열하고 TO-BE 를 추천받아 1:1 로 정해 두면, 반복해서 나오는 컬럼이 Auto Map 에서 한 번에 맞춰짐. 변환식(CASE · 코드 변환 등)도 함께 적어 두면 Auto Map 이 별칭만 그 테이블에 맞게 바꿔 넣음
- **SQL 방식 매핑** — 화면으로 표현하기 어려운 집계 · 가공은 PL/pgSQL 을 직접 기술하되, 동일한 실행 · 기록 체계로 관리
- **Before / After Job** — 적재 전후에 수행할 문장 (시퀀스 재시작, 후처리 등)



Column Map — 대상 테이블 tb_cust(고객)의 컬럼 15개를 왼쪽에, 각 컬럼이 어느 소스 테이블의 어느 컬럼에서 오는지를 오른쪽에 나란히 둡니다. 소스는 cu_cust(A1) · cu_cust_dtl(A2) · cu_cust_addr(A3) 세 테이블을 조인한 것이고, 맨 오른쪽 매핑 로직에 변환식이 SQL 로 적혀 있습니다 — 길이 맞춤(`left`), 코드 변환 테이블 조회(`coalesce((SELECT … FROM 변환 테이블 …))`), 문자열 날짜 → timestamp(`udf_char2date`), 주소 합치기(`nullif(concat_ws(' ', A3.addr1, A3.addr2), '')`), 프로젝트 공용 변수(`${DEFAULT_SYSTEM_DTM}`). 컬럼 앞의 점은 매핑 상태이며, 회색(확인 필요)은 원본이 대상보다 긴 경우처럼 매핑 단계에서 미리 살펴야 할 컬럼입니다(예: email — 원본 character varying(100), 대상 character varying(60)). 이 화면이 곧 매핑 정의서입니다.



Mapping Dictionary — AS-IS 컬럼과 TO-BE 컬럼의 1:1 대응표(용어사전)입니다. Refresh 를 누르면 원본 테이블의 컬럼을 이름별로 세어 빈도 분포를 보여 주고, 선택한 빈도 이상인 컬럼을 많이 나오는 순서로 배치합니다 — 위 예는 업무영역 ALL, 빈도 2 이상인 11개입니다(cust_no 는 원본 테이블 5개에 나온다). Auto Match 는 비어 있는 TO-BE 칸을 매핑중 → 한글명 → 유사도 순서로 추천해 채우고, 후보가 여럿이면 점수와 근거를 붙인 콤보로 선택하게 합니다(reg_dt — "(0.95) created, 등록일시 매핑중 1"). 여기서 정해 둔 짝은 Column Map 의 Auto Map 이 가장 먼저 쓰므로, 테이블마다 같은 컬럼을 다시 선택할 필요가 없습니다. 빈도 · 매핑 수는 Refresh · Save 때만 세어, AS-IS 컬럼이 수만 개여도 화면은 바로 열립니다. 변환식 칸에 CASE 같은 식을 적어 두면 Auto Map 이 그 식을 매핑 로직에 넣고, 원본 별칭(A1 등)은 테이블마다 맞게 바꿉니다. 변환식이 없으면 원본 컬럼만 붙입니다.



Variables — 프로젝트 공용 변수입니다. 업무영역마다 이름 · 값 · 설명을 두고, 매핑 로직에서 `${이름}` 으로 씁니다(앞의 Column Map 에 보이는 `${DEFAULT_SYSTEM_DTM}` — 값 `localtimestamp(0)`). 기준 일시 · 시스템 코드처럼 여러 테이블이 함께 쓰는 값을 한 곳에서 정하고, 값을 바꾸면 스크립트를 다시 생성하는 것으로 모든 매핑에 반영됩니다.

5.3 적재 방식 (대상 테이블별 지정)

- TRUNCATE 여부, 병렬도 — 병렬도를 2 이상으로 두면 메인 소스를 ctid 범위로 나눠 여러 세션(dblink)이 동시에 적재. 파티션 테이블은 맨 아래 파티션마다, 크면 일감 크기(CHUNK_SIZE_MB)씩 나눈다
- **인덱스 · 제약 · 트리거 분리와 복구** — 적재 전에 떼어 낸 정의를 리포지토리 DB 에 보관하고(KEEP_*), 적재 뒤 원래 정의 그대로 복구
- MERGE 적재(PostgreSQL 15 이상), **중복 행 제거**(제거한 행은 별도 스키마에 보관), 선이행 대상 지정
- 힌트 — Table Map 에 적은 힌트는 pg_hint_plan 이 설치된 DB 에서 읽히고, 없으면 주석으로 남는다

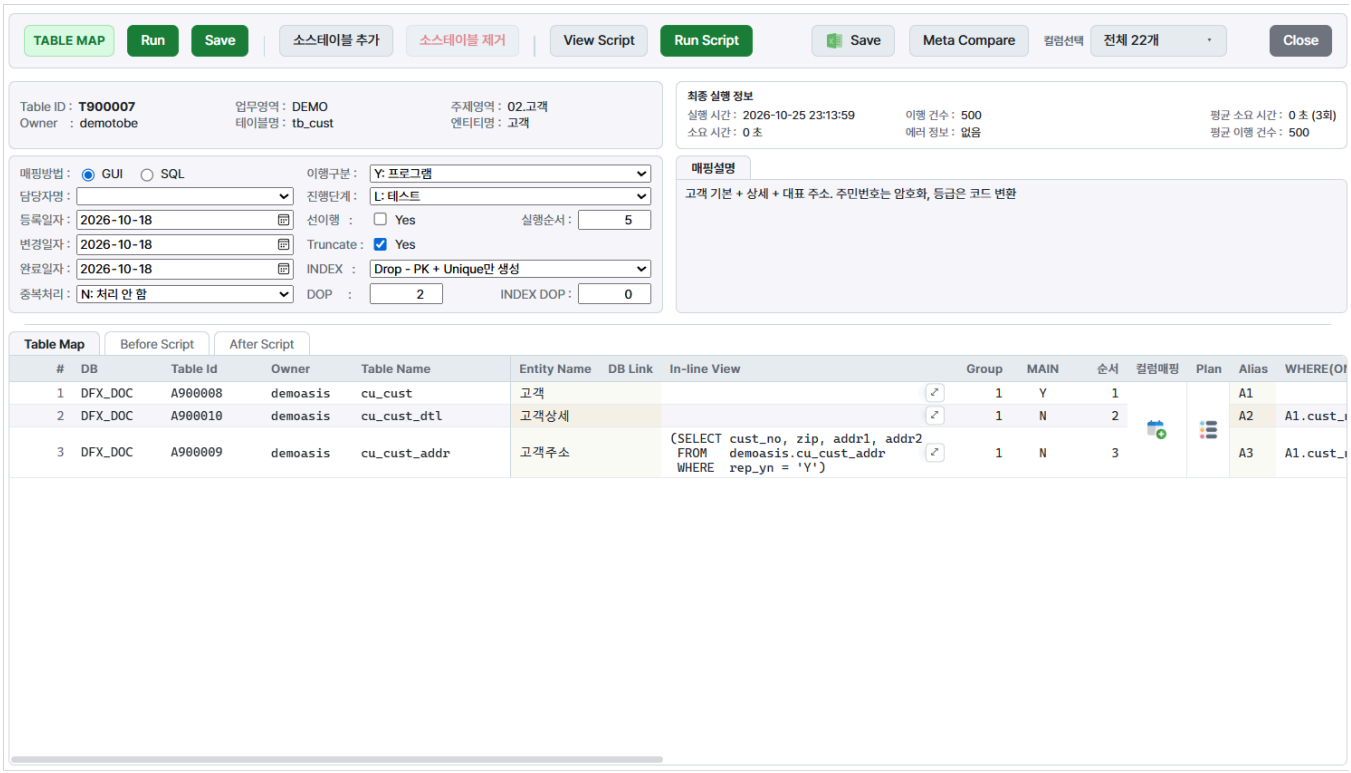


Table Map — 대상 테이블 하나(tb_cust)의 이행 방법을 한 화면에서 정합니다. 위쪽이 적재 옵션입니다 — Truncate · 인덱스 처리(Drop — PK + Unique 만 생성) · 병렬도(DOP) · 중복 처리 · 선이행 · 실행 순서, 그리고 이행 구분(프로그램 / 엑셀 / 부분 셋업 / 이행 안 함) · 진행 단계 · 담당자. 아래 표가 소스 묶음입니다 — cu_cust(A1, MAIN)에 cu_cust_dtl(A2)을 LEFT JOIN 하고, cu_cust_addr 는 대표 주소만 선택하는 인라인 뷰(`WHERE rep_yn = 'Y'`)로 A3 에 붙였습니다. 조인 조건은 셀에 SQL 로 적고, Before / After Script 탭에 적재 전후 문장을 둡니다. 오른쪽 위 "최종 실행 정보" 는 이 테이블의 마지막 실행(건수 · 소요 시간 · 오류)과 평균(3회)으로, 매핑 화면에서 리허설 결과가 바로 보입니다.

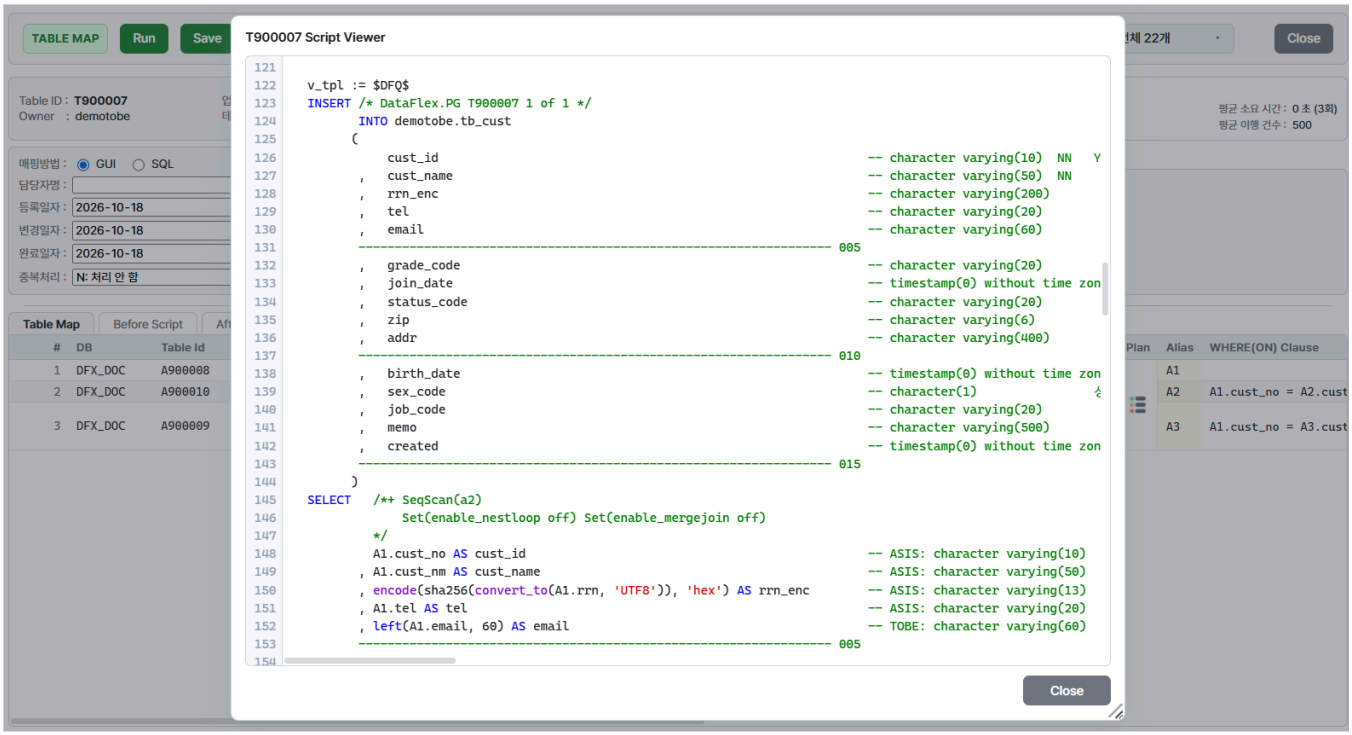


Table Map · Script Viewer — 파일로 저장하기 전에, 지금 매핑으로 만들어질 스크립트를 매핑 화면 안에서 바로 봅니다(tb_cust, T900007). 대상 컬럼 15개의 INSERT 목록과 SELECT 힌트, 컬럼마다 붙는 타입 · NN · PK · 속성명 주석이 §6 의 파일과 같습니다. 옵션 하나를 바꾸고 결과를 확인하는 데 파일을 열 필요가 없습니다.

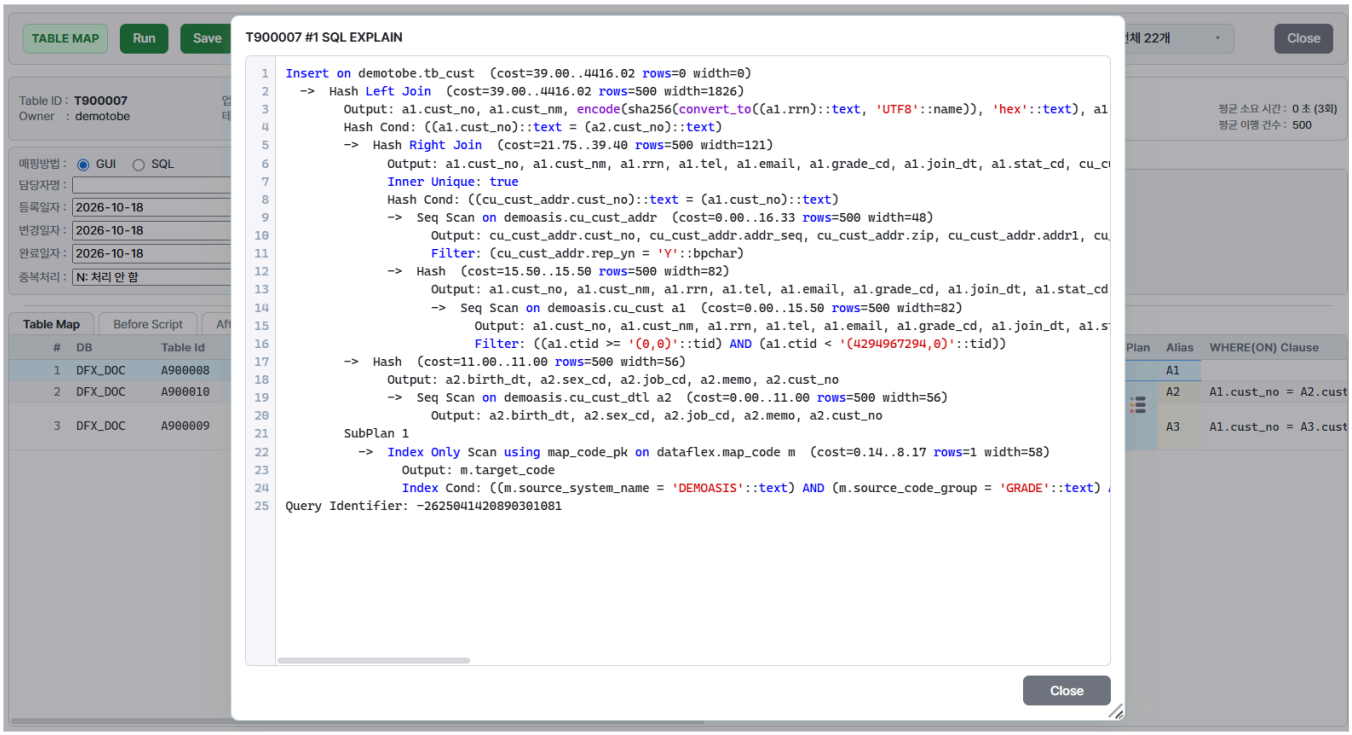
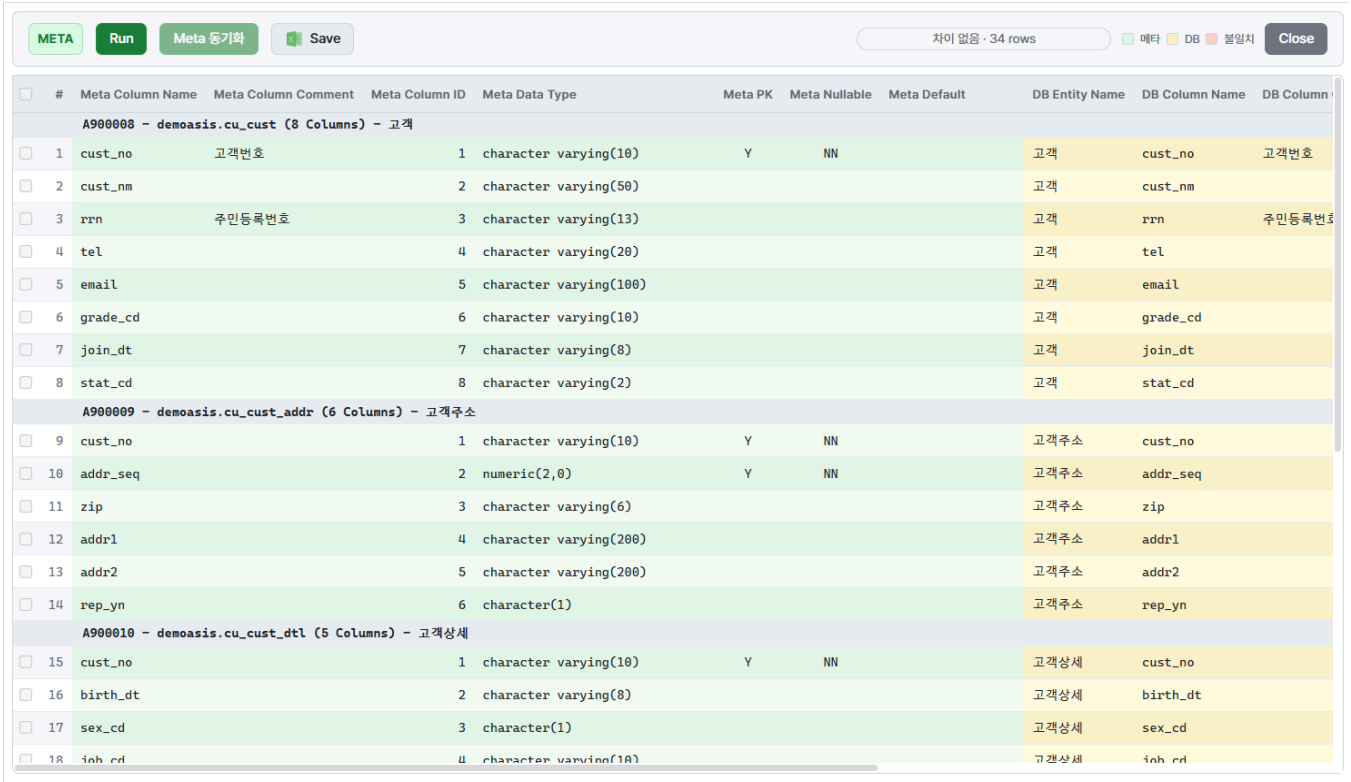


Table Map · SQL EXPLAIN — 같은 자리에서 그 INSERT … SELECT 의 실행 계획을 봅니다(EXPLAIN VERBOSE). 세 소스의 Hash Join, 코드 변환 테이블의 Index Only Scan, 예상 건수(rows=500)와 비용, 병렬 조각의 ctid 범위 조건이 보입니다. 조인 방식이나 병렬도를 바꿔야 할지 리허설을 돌리기 전에 계획에서 먼저 판단합니다.

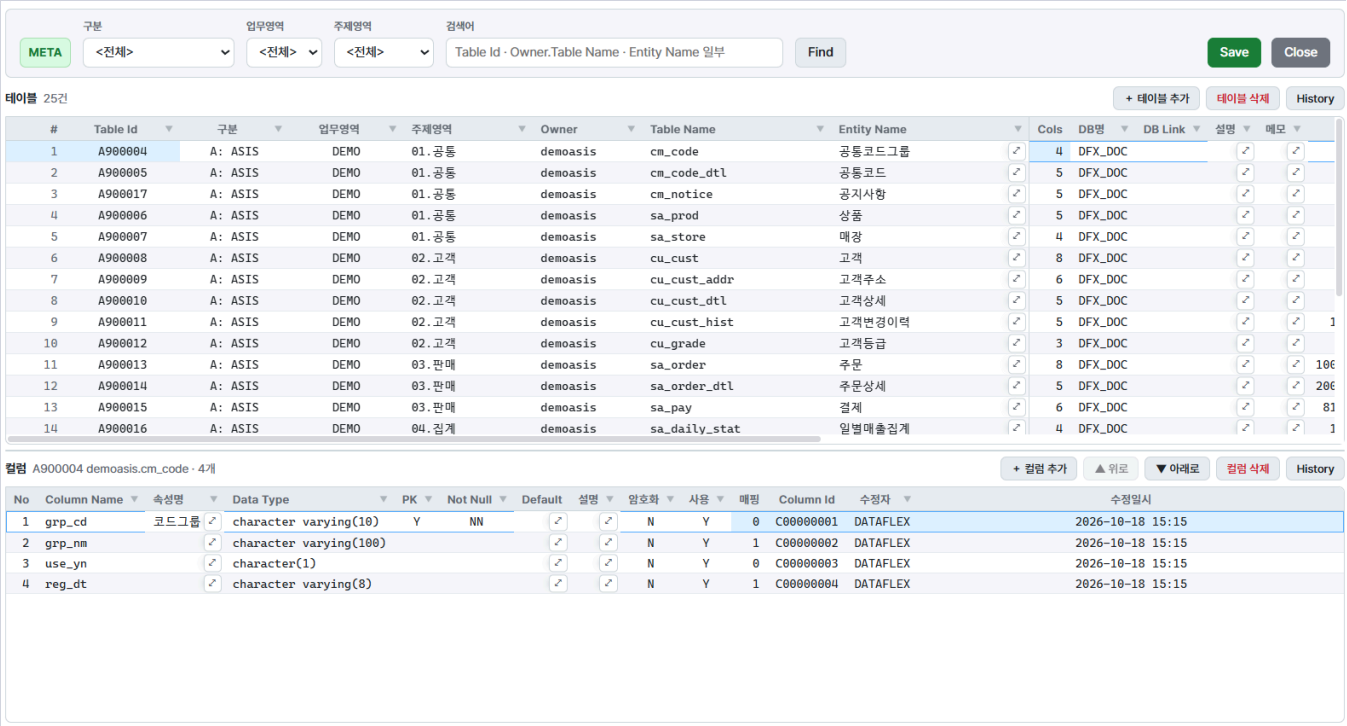
5.4 메타 · 변경 관리

- **Meta Compare** — 리포지토리 DB 에 등록된 메타 정보와 실제 DB 의 차이를 전부 표시하고, 컬럼 · 테이블 · 전체 단위로 **선택하여** 반영
- **변경 이력** — 매핑 · 메타의 변경을 DB 트리거로 기록. 변경자 · 시각 · 내용을 화면에서 조회
- **이행 구분 · 진행 단계** — 테이블별로 프로그램 이행 / 엑셀 / 부분 셋업 / 이행 안 함을 구분하고, 미진행 · 개발 · 테스트 · 완료 단계를 관리

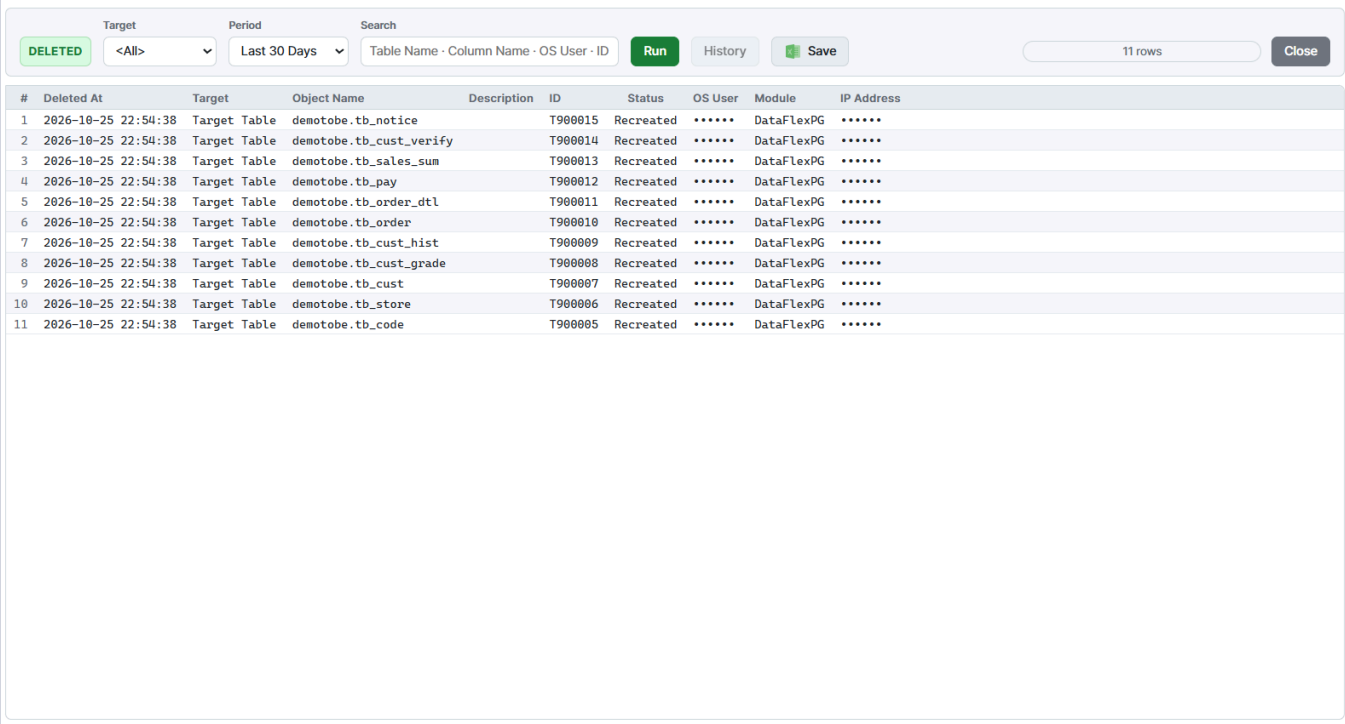


Meta Compare — 리포지토리 DB 에 등록된 메타(왼쪽, 초록)와 실제 DB 의 지금 구조(오른쪽, 노랑)를 컬럼 한 줄씩 나란히 놓고 비교합니다. 위 예는 대상 tb_cust 의 소스 세 테이블(cu_cust · cu_cust_addr · cu_cust_dtl) 등 컬럼 34개를 한 번에 본 것이고, 오른쪽 위에 "차이 없음 · 34 rows" 로 결론이 먼저 나옵니

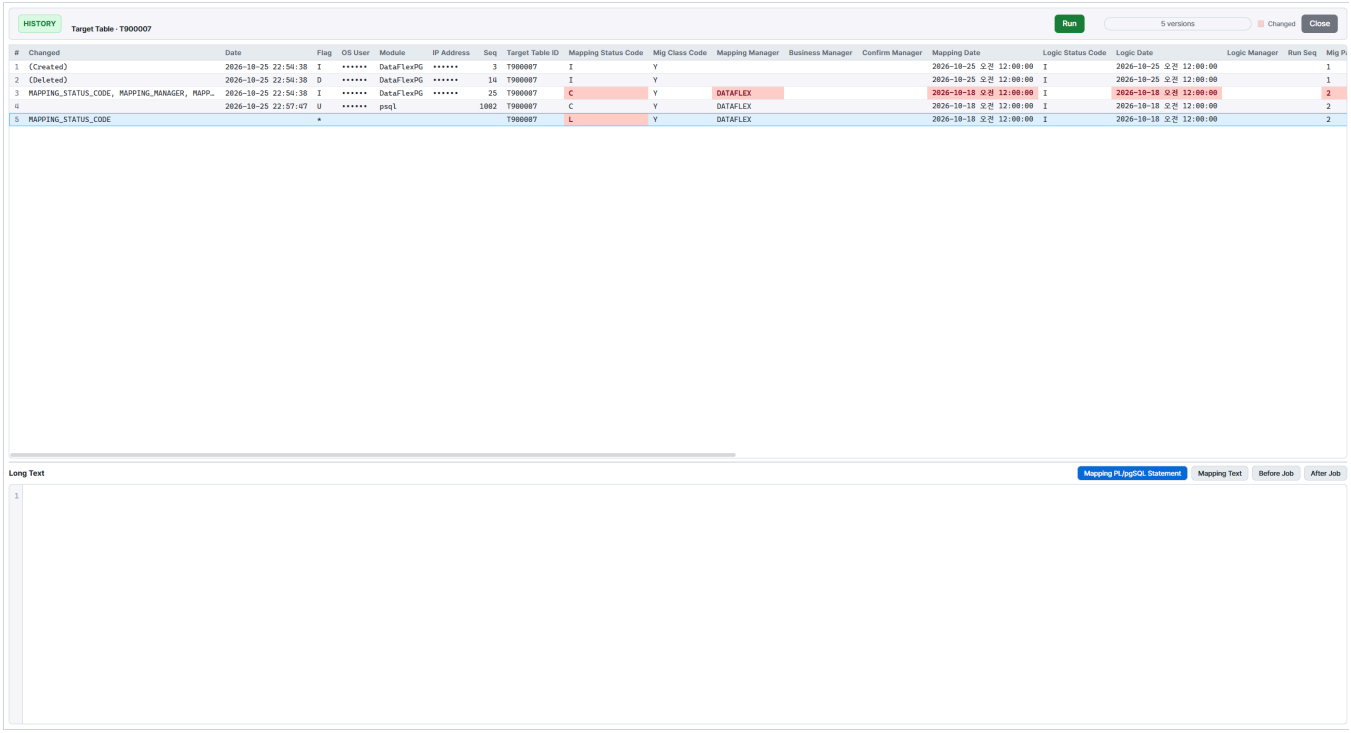
다. 원본 시스템에 컬럼이 늘거나 길이가 바뀌면 그 줄만 색이 달라지고(메타에만 · DB 에만 · 값 다름), 필요한 줄만 선택해 Meta 동기화로 메타에 반영합니다 — 무엇을 더하고 지우고 고칠지 먼저 보여 주고 묻습니다. 리허설 사이에 "원본이 바뀌지 않았나" 를 확인하는 화면이며, 결과는 엑셀로도 저장합니다.



Meta Editor — 등록된 메타(테이블 · 컬럼 정보)를 수작업으로 고치는 화면입니다. 위 그리드는 조건에 맞는 테이블, 아래 그리드는 선택한 테이블의 컬럼이며, 키와 자동값(등록 · 수정 일시, 수정자)만 빼고 전부 고칠 수 있습니다. Save 는 두 그리드의 변경분을 한 트랜잭션으로 올리고, 테이블을 지울 때는 딸려 지워질 매핑 · 이행 로그의 건수를 보여 주고 확인을 받은 뒤 지웁니다. 지운 것은 Meta Delete History 에 남습니다.



Meta Delete History — 이력 테이블 다섯 개에서 지워진 기록만 모아 시간 역순으로 봅니다. 다른 화면은 살아 있는 행만 보여 주므로 "누가 언제 무엇을 지웠나" 는 여기서만 확인 가능합니다. 지운 뒤 같은 키로 다시 만든 것은 Recreated 로 표시합니다. 지워진 테이블 · 컬럼의 이름은 이력에서 되찾아 보여 주고, 행을 선택하면 변경 이력 창이 열려 지워지기 직전까지 무엇이 바뀌었는지 이어 볼 수 있습니다.



메타 정보와 매핑의 변경 이력을 버전별로 표시합니다. 변경 일시, 구분(I: 등록 · U: 수정 · D: 삭제), OS 사용자, 모듈, IP 주소를 확인할 수 있으며, 변경된 값은 색으로 구분합니다. 화면 밖에서 SQL 로 바꾼 것도 트리거가 기록합니다(위 4번 — 모듈 psql). 매핑 PL/pgSQL, 매핑 설명, Before, After Job 등 긴 텍스트는 화면 아래쪽에서 확인합니다.

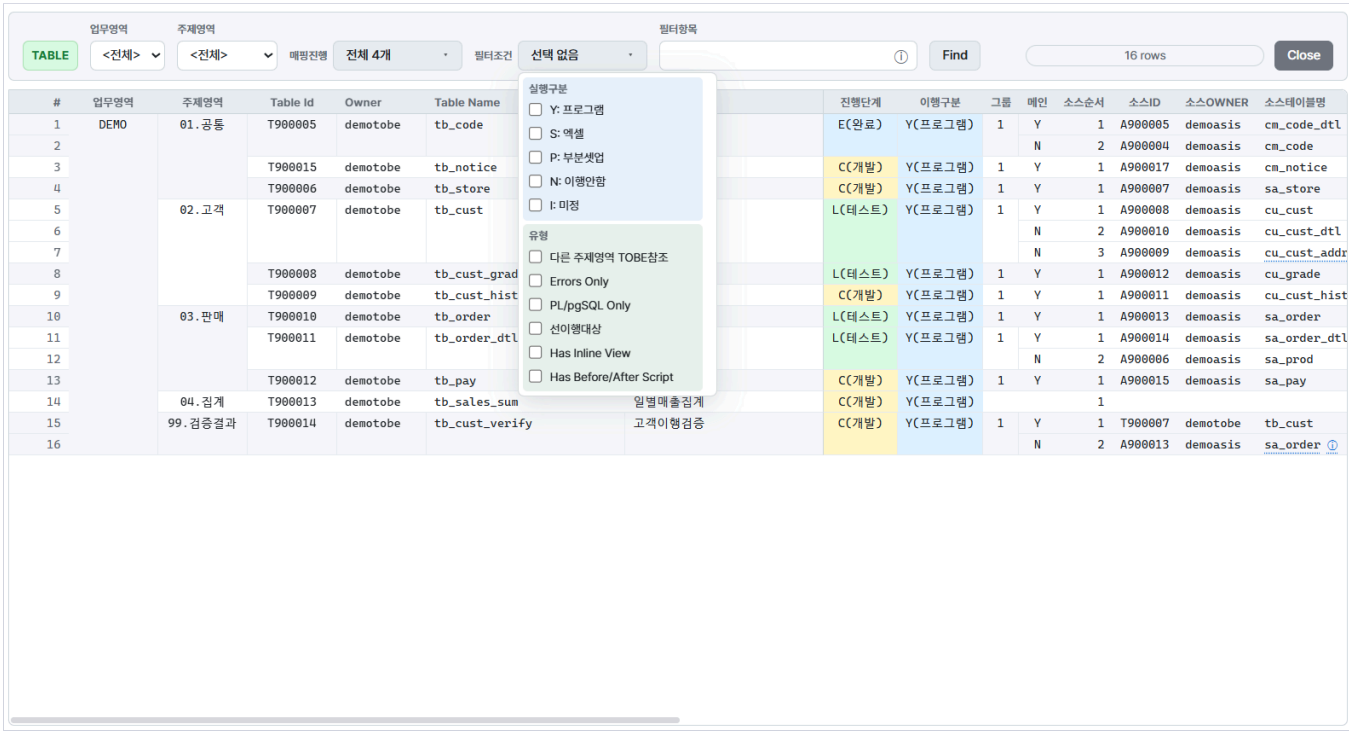
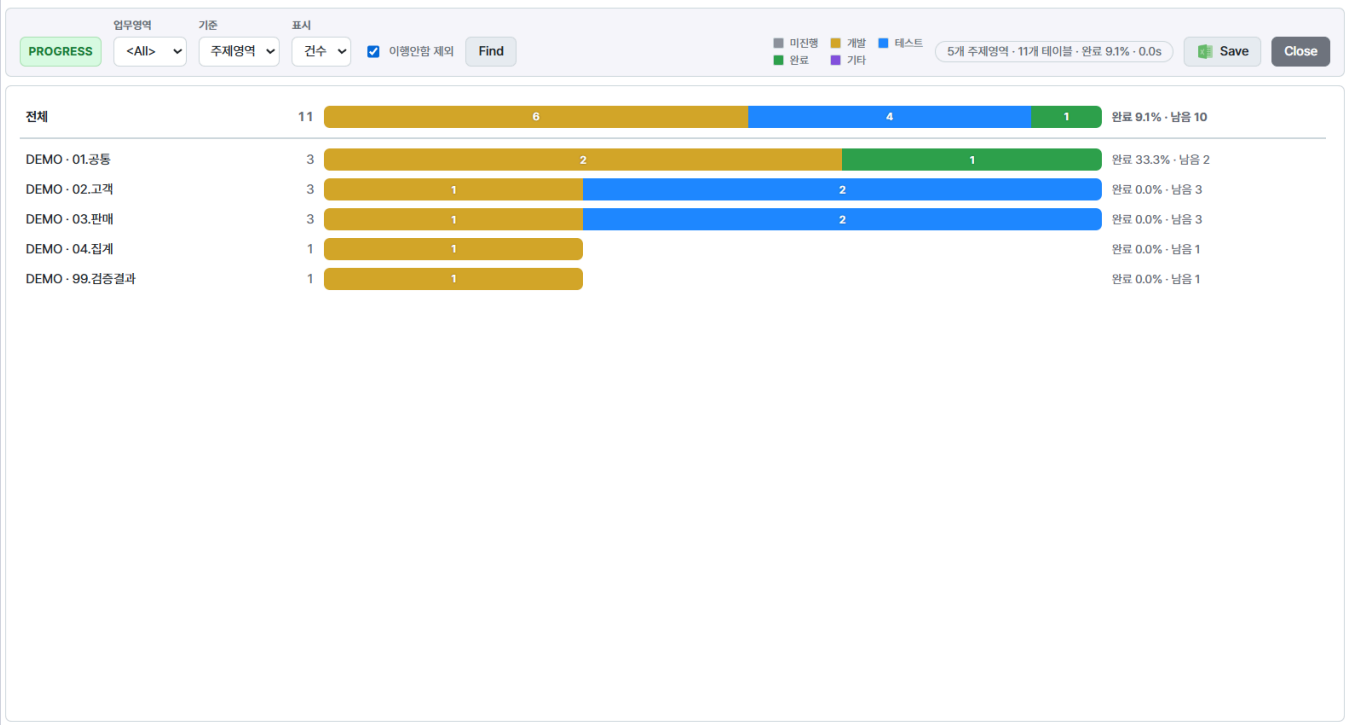


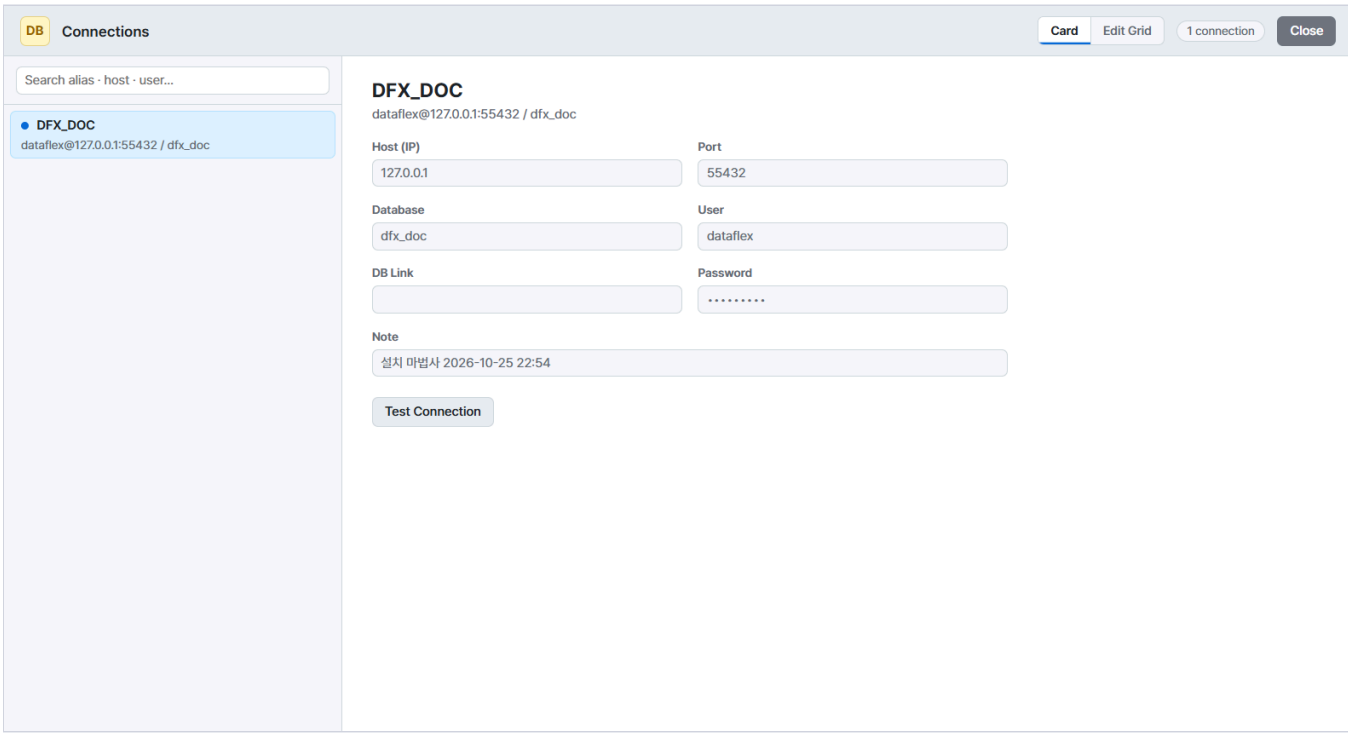
Table List — 대상 테이블 전체를 소스 테이블과 함께 한 줄씩 늘어놓은 목록입니다(주제영역 · 진행 단계 · 이행 구분 · 소스 순서 · DB 링크 · Truncate · 인덱스 제거 / 복구 · 마지막 실행 시각). 담당자가 "지금 어디까지 됐나" 를 보는 첫 화면이며, 열어 둔 필터처럼 이행 구분(프로그램 / 엑셀 / 부분 셋업 / 이행 안 함 / 미정)과 유형(다른 주제영역 TOBE 참조 · 오류만 · PL/pgSQL 방식 · 선이행 · 인라인 뷰 · Before / After Script 있음)으로 거르고, 매핑 진행 단계(미진행 / 개발 / 테스트 / 완료)와 검색어(최근 검색 기억)로도 좁힙니다.



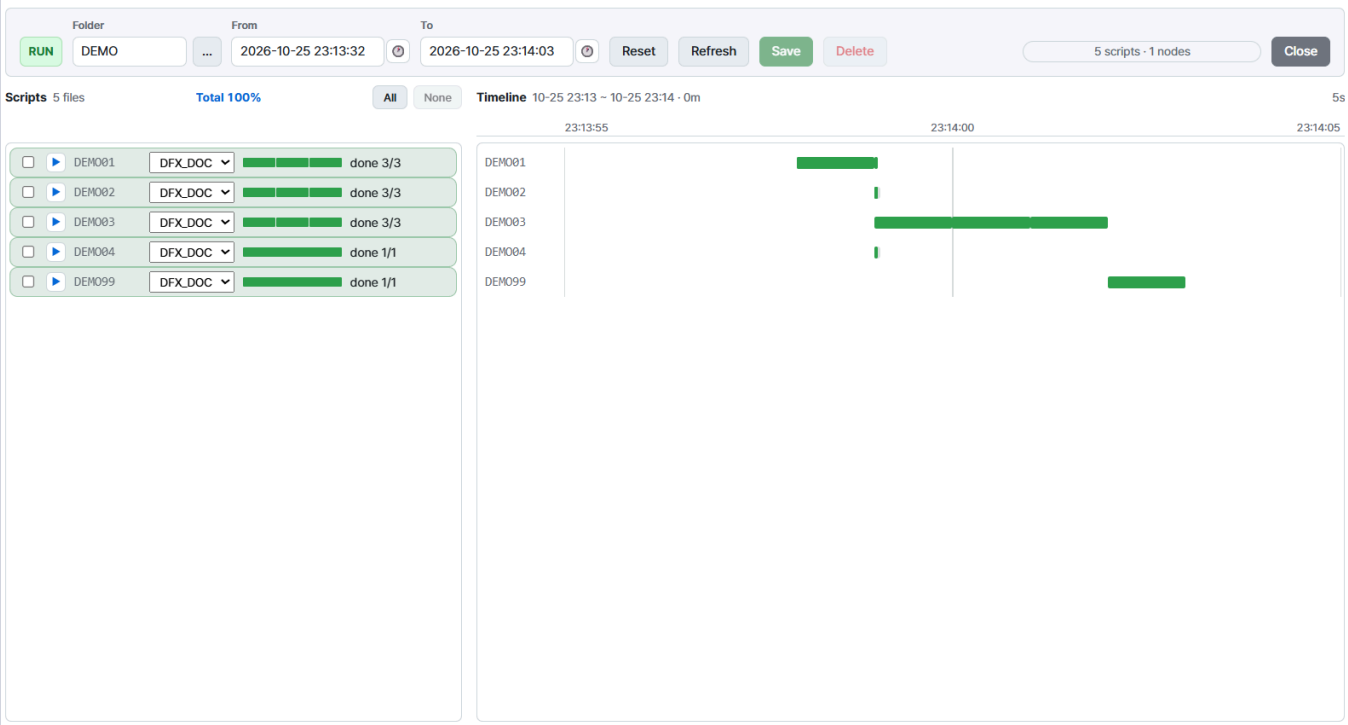
Mapping Progress — 진행 단계(미진행 · 개발 · 테스트 · 완료)를 주제영역별 누적 막대로 집계한 화면입니다. 위 예는 대상 11개 테이블 가운데 개발 6 · 테스트 4 · 완료 1(완료 9.1 %, 남음 10). 주제영역 대신 담당자로 묶거나 건수 대신 비율로 볼 수 있고, 이행하지 않는 테이블은 뺍니다. 막대 조각을 누르면 그 칸에 든 테이블 목록이 아래에 펼쳐집니다. 매핑 화면에서 단계를 바꾸면 여기에 바로 반영되므로, 진척 보고를 위해 따로 집계하지 않습니다.

5.5 실행 · 기록

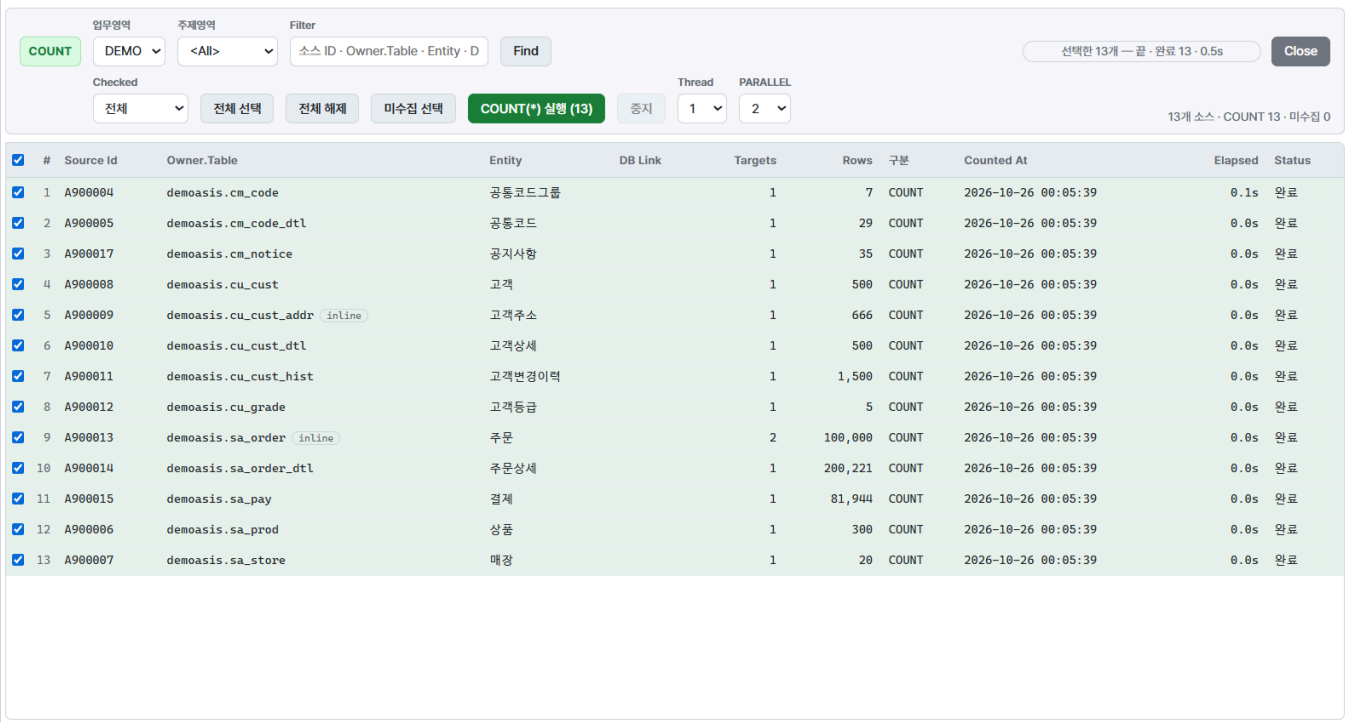
- 실행 노드별 배정과 스크립트 · 테이블 단위 타임라인
- **개별 테이블이 실패해도 기록을 남기고 다음 테이블을 계속 수행** — 야간 리허설이 첫 오류에서 중단되지 않음. 실행 기록은 별도 세션(dblink 자기 연결)으로 따로 커밋되므로, 적재가 롤백되어도 기록은 남음
- **건수 대사** — 적재 건수와 "소스에서 나와야 할 건수" 를 테이블마다 비교해 일치 · 불일치를 판정. 예상 건수는 이행 스크립트와 같은 조인 · 조건으로 도구가 센다
- 화면의 목록은 대부분 엑셀로 저장 가능



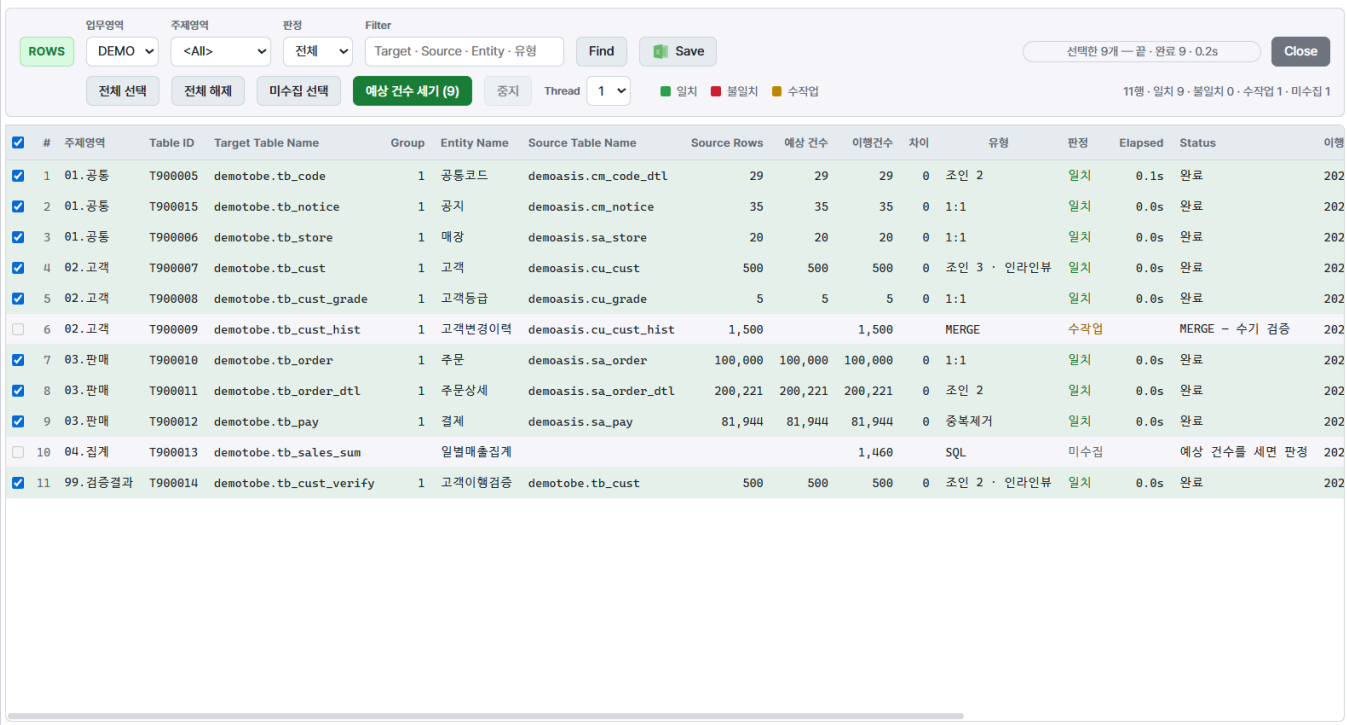
Run Connections — Script Launcher 가 루트 스크립트를 돌릴 실행 노드의 목록입니다. 도구가 여는 접속(Tool Connections)과 같은 모양이지만 목록을 따로 두어 이행 대상 목록에 노드가 섞이지 않게 합니다. Launcher 화면에서 스크립트마다 이 노드 중 하나를 선택해 배정합니다. 비밀번호는 그림에서 가렸습니다.



Script Launcher — 왼쪽은 이 폴더(DEMO)의 루트 스크립트 5개와 각각을 돌릴 실행 노드(DFX_DOC)의 배정, 진행률(done 3/3 = 그 루트 아래 테이블 스크립트 3개 중 3개 완료)입니다. 오른쪽 타임라인은 스크립트마다 언제 시작해 언제 끝났는지를 한 줄로 그려, 어디서 시간이 가는지 한눈에 보입니다. 위 예는 검증용 샘플(대상 11개 테이블)을 DEMO01 다음에 DEMO02 · 03 · 04 를 동시에 돌린 것으로 전체 5초, Total 100%. 스크립트는 프로그램 옆 psql 폴더의 psql 로 실행됩니다. 개별 테이블이 실패해도 기록을 남기고 다음 테이블을 계속 수행하므로, 야간 리허설 뒤 아침에 이 화면에서 결과를 확인합니다.



Source Count — 매핑의 소스로 쓰이는 원본 테이블의 건수를 `COUNT(*)` 로 세어 메타에 넣는 화면입니다. 등록할 때 들어가는 통계값은 근사치라 검수에 못 쓰므로 실제 건수로 덮고 센 시각을 남깁니다 — 이 값이 건수 대사의 기준이 됩니다. 무엇을 셀지는 사람이 선택하고, 여러 테이블을 백그라운드에서 동시에 세며 줄마다 상태가 바로 보입니다(13개 · 0.5초). 세는 길은 이행 스크립트와 같습니다.



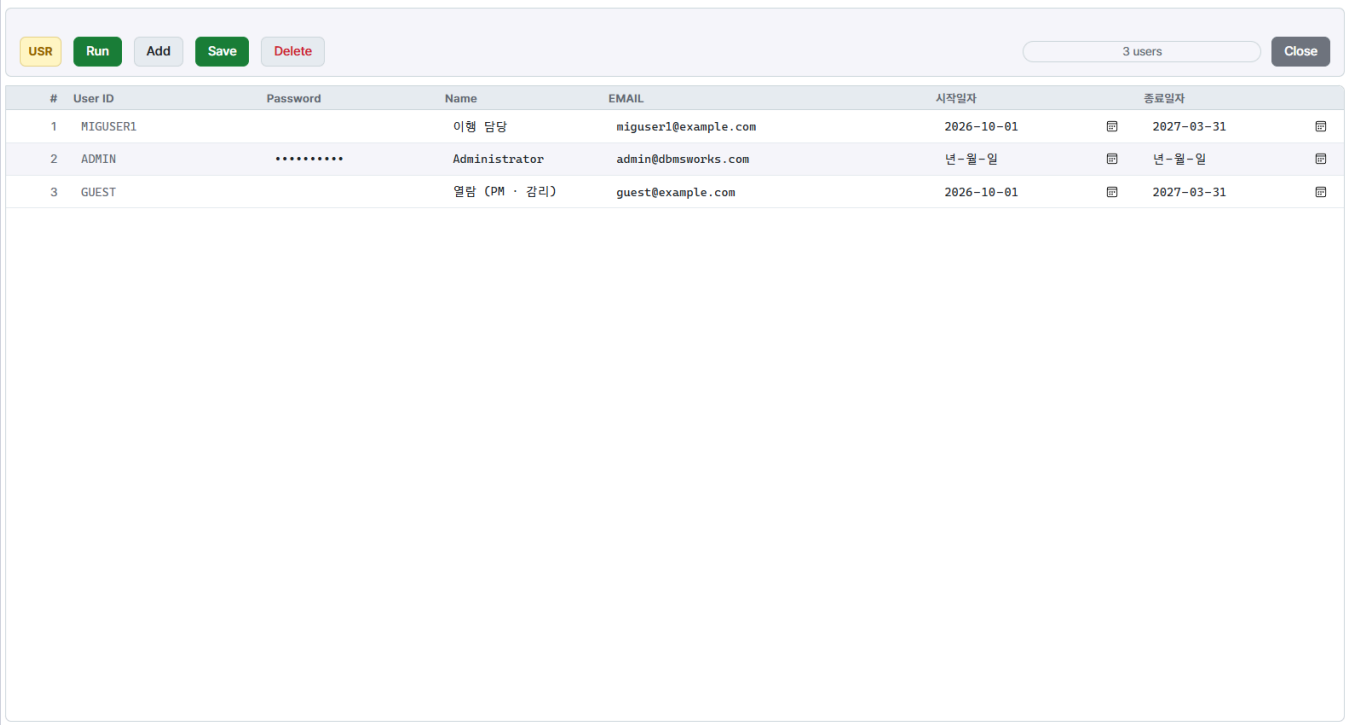
*Rows Compare(건수 대사) — 적재가 끝난 뒤 "소스에서 나와야 할 건수가 다 들어왔나" 를 테이블마다 판정하는 화면입니다. 한 줄이 대상 테이블 하나(와 그 메인 소스)이고, Source Rows(소스 건수) · 예상 건수 · 이행건수(INSERT 된 행 수) · 차이 · 유형 · 판정이 나란히 보입니다. 위 예는 DEMO 11개 테이블 가운데 9개를 "예상 건수 세기" 로 센 직후(0.2초)로 일치 9 · 수작업 1(MERGE) · 미수집 1(SQL 방식)입니다. 소스 하나를 그대로 옮기는 1:1 매핑은 Source Count 의 소스 건수로 바로 판정하고, 조인 · 인라인 뷰 · 조건이 걸린 매핑은 **이행 스크립트와 같은 FROM · 조인 · 조건 · GROUP BY 로 도구가 COUNT 문을 만들어** 예상 건수를 셉니다 — 대상 테이블을 `COUNT(*)` 하는 것이 아니라, 스크립트가 읽었을 건수를 다시 세는 것입니다. 무엇을 셀지 선택하고 백그라운드에서 동시에 세며, 결과는 리포지토리 DB 에 남아 다음에 열어도 그대로입니다. tb_pay 처럼 중복 제거가 걸린 테이블도 INSERT 건수와 예상 건수가 같으면 일치로 봅니다 — 중복 제거는 적재 뒤의 일이라 판정 밖입니다. 적재 뒤에 매핑이 바뀐 줄은 "매핑 변경" 으로 표시되며, 불일치만 걸러 보거나 엑셀로 저장합니다.*

5.6 읽기 전용 열람

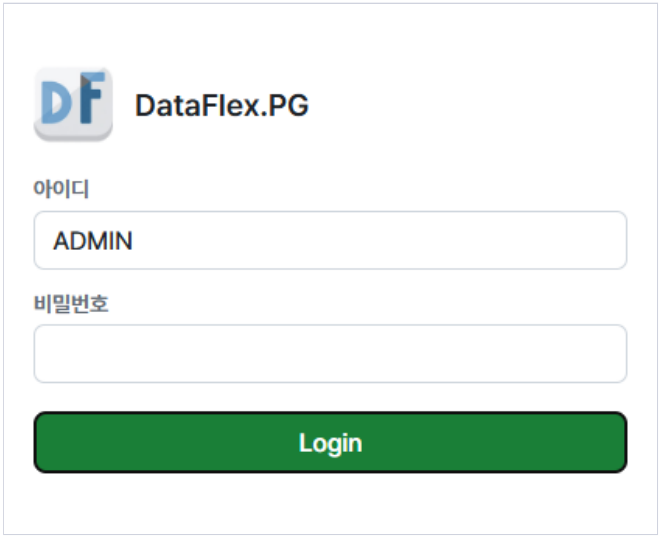
현업 담당자 · PM · 감리가 **수정 권한 없이** 매핑 정의, 진행 현황, 실행 기록을 직접 조회할 수 있습니다. 현황을 확인하려고 이행 담당자에게 자료를 요청할 필요가 없고, 열람 중 실수로 매핑이 바뀔 위험도 없습니다.

구분	동작
읽기 전용 계정(GUEST)	모든 화면을 조회만 가능. 편집 · 저장 · 스크립트 실행 불가
일반 사용자	메타 등록 · 매핑 · 스크립트 생성 · 실행 순서 · 원본 건수 집계는 가능. 접속 정보 · 사용자 · 옵션 · 공용 변수 관리와 **이행 실행(Script Launcher)** 은 조회만 가능
관리자	전체 기능 — 이행 실행과 프로젝트 공통 설정은 관리자만 변경
사용 기간 · 라이선스가 끝난 경우	접속과 조회는 유지되고 편집 · 실행만 제한 — 기간이 끝나도 기록은 계속 확인 가능 (라이선스는 5.7 참조)

읽기 전용일 때는 해당 단추와 입력 칸이 **처음부터 비활성**으로 표시되며(눌렀다가 거절당하는 방식이 아님), 화면 뒤의 처리 단계에서도 한 번 더 차단합니다. 제한된 이유(읽기 전용 계정 / 사용 기간 / 관리자 아님)를 구분하여 안내합니다.



Users — 도구 사용자 목록입니다. 아이디 · 이름 · 메일 · 사용 기간(시작 · 종료)을 관리자가 등록하고, 사용자마다 구분(관리자 / 일반 / 읽기 전용)을 둡니다. 비밀번호는 리포지토리 DB 에 암호화(AES-GCM)되어 저장됩니다(그림에서는 가렸습니다). 사용 기간이 끝나면 접속과 조회는 되고 편집 · 실행만 막힙니다.



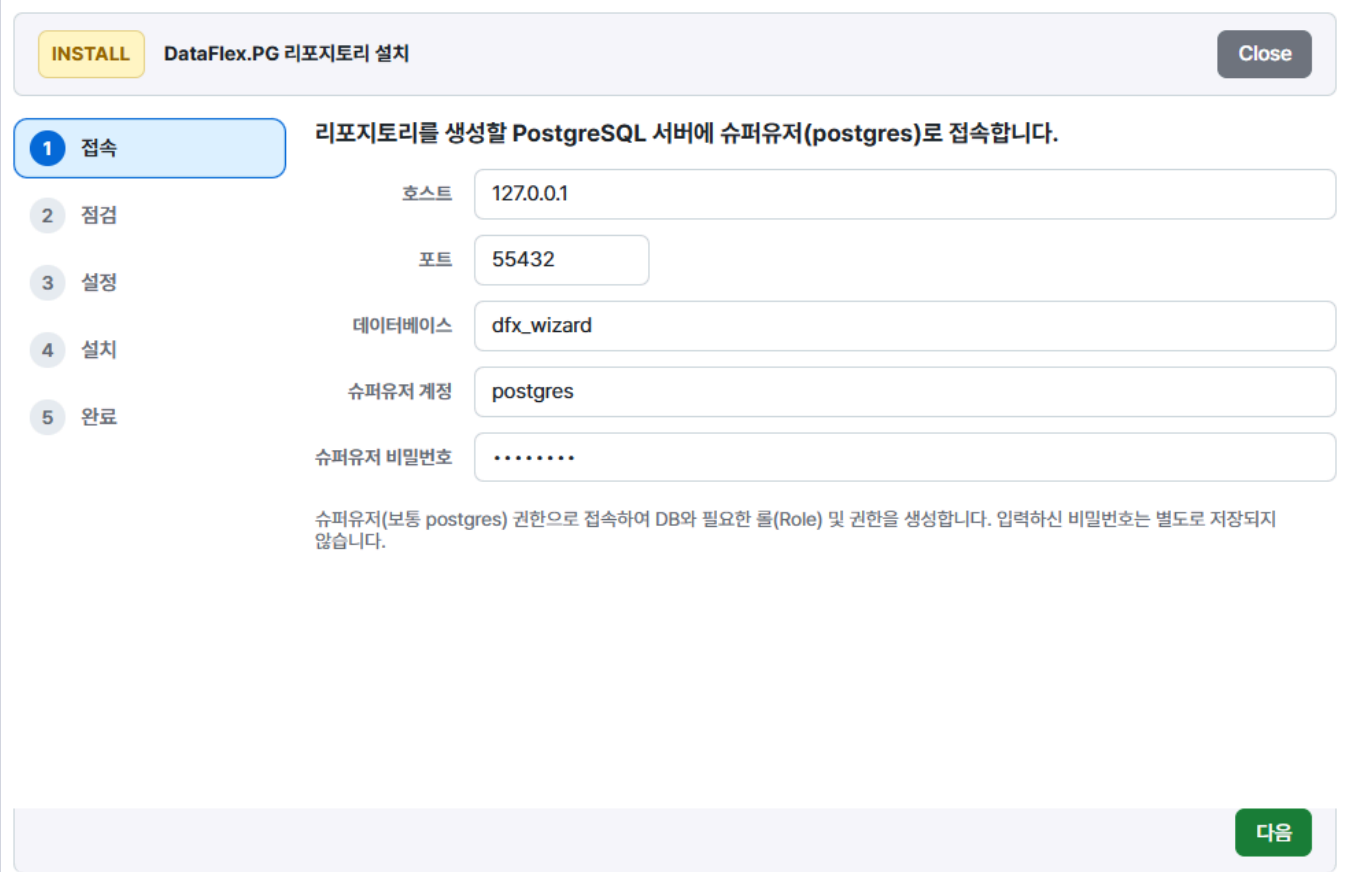
관리자가 등록한 아이디로 로그인합니다. 최초 로그인 시 비밀번호를 설정합니다. 화면 문구는 한국어가 기본이며, Options → Language 에서 영어로 바꿀 수 있습니다(다시 시작하면 적용).

5.7 설치 · 라이선스 · 데이터 초기화

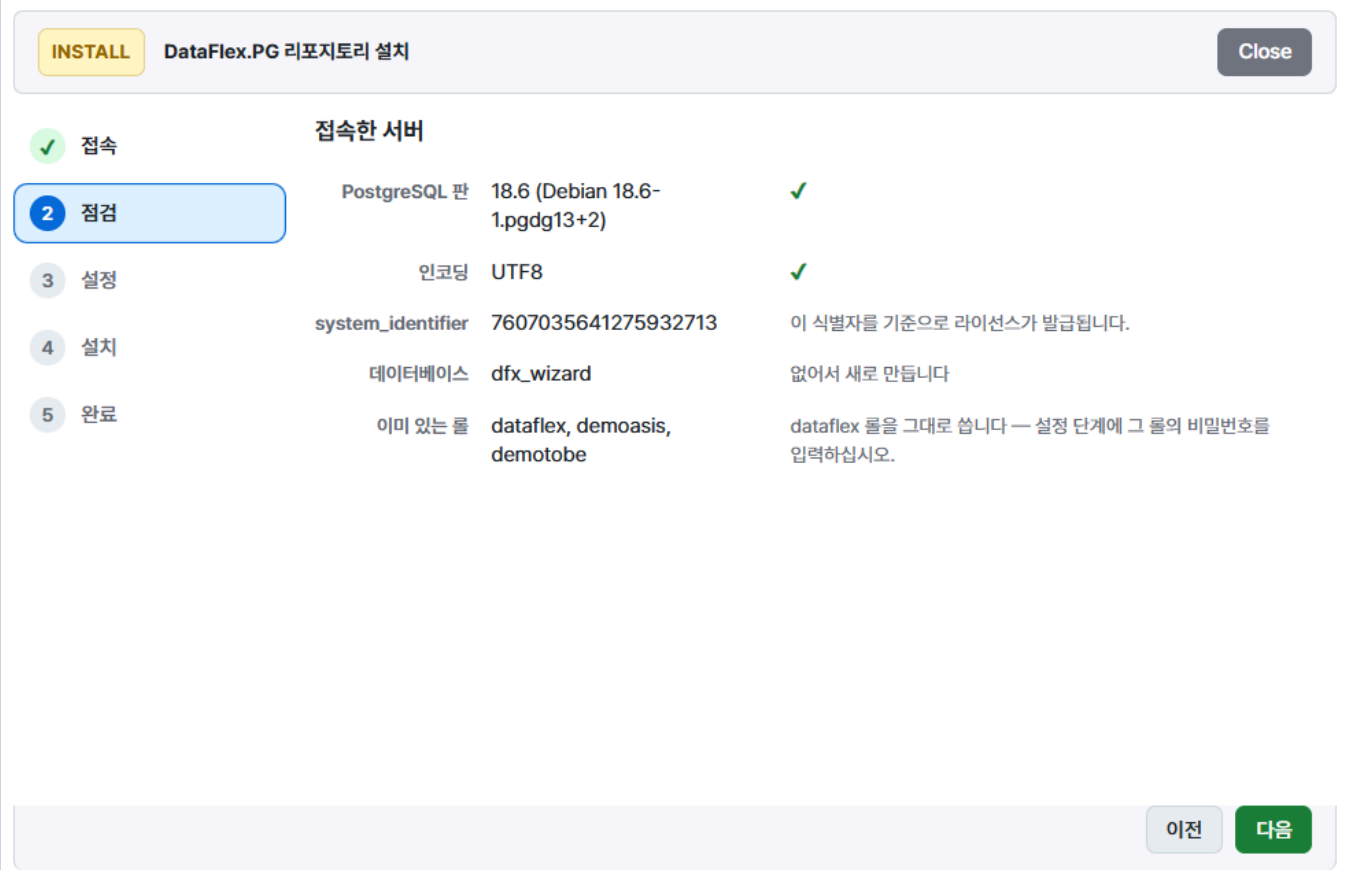
리포지토리 DB 는 프로그램에 들어 있는 **설치 마법사**로 설치합니다. `DataFlexPG.exe -install` 로 실행하면 접속 → 점검 → 설정 → 설치 → 완료 순서로 진행되며, 설치 스크립트가 실행 파일 안에 들어 있어 따로 받을 필요가 없습니다.

단계	내용
접속	리포지토리를 생성할 PostgreSQL 서버에 슈퍼유저(postgres)로 접속합니다. 입력하신 비밀번호는 저장되지 않습니다.
점검	PostgreSQL 버전(14 이상) · 인코딩 · system_identifier · 이미 있는 롤 · 데이터베이스 유무를 확인합니다. 그 데이터베이스에 리포지토리가 이미 있으면 설치하지 않습니다.
설정	접속 이름 · dataflex 비밀번호 · 데모 스키마 설치 여부 · 인증 키를 정합니다.
설치	데이터베이스(없으면 UTF8 로 새로) → 롤 · 스키마 · 확장(dblink · postgres_fdw) → 테이블 · 함수 · 뷰 · 트리거 → 기초 데이터(용어 사전 · 옵션 · 변수) → (선택) 데모 스키마 순서로 설치합니다.
완료	접속 정보(Tool · Run)와 관리자 계정 ADMIN 을 등록하고, 실행 폴더에 설정 파일(DataFlexPG.db)을 만듭니다.

PostgreSQL 의 롤은 서버 전체의 것이라, 같은 서버의 다른 데이터베이스에 이미 설치해 dataflex 롤이 있으면 그 롤을 그대로 씁니다(설정 단계에서 그 롤의 비밀번호를 입력). 설치 마지막에 실행 기록용 자기 연결(dataflex_self)로 실제로 붙어 보고, 붙지 못하면 설치를 중단합니다.



설치 마법사의 접속 단계입니다. 리포지토리를 생성할 서버의 호스트, 포트, 데이터베이스 이름과 슈퍼유저 계정 · 비밀번호를 입력합니다. 입력한 비밀번호는 저장되지 않습니다.



점검 단계에서는 접속한 서버의 PostgreSQL 버전, 인코딩, system_identifier, 데이터베이스 유무 및 이미 있는 롤을 확인합니다. 설치 조건을 충족하지 못하면 다음 단계로 진행할 수 없습니다.



설치 마법사의 설정 단계 — 인증 키는 넣는 즉시 서명 · system_identifier · 만료일을 확인합니다.

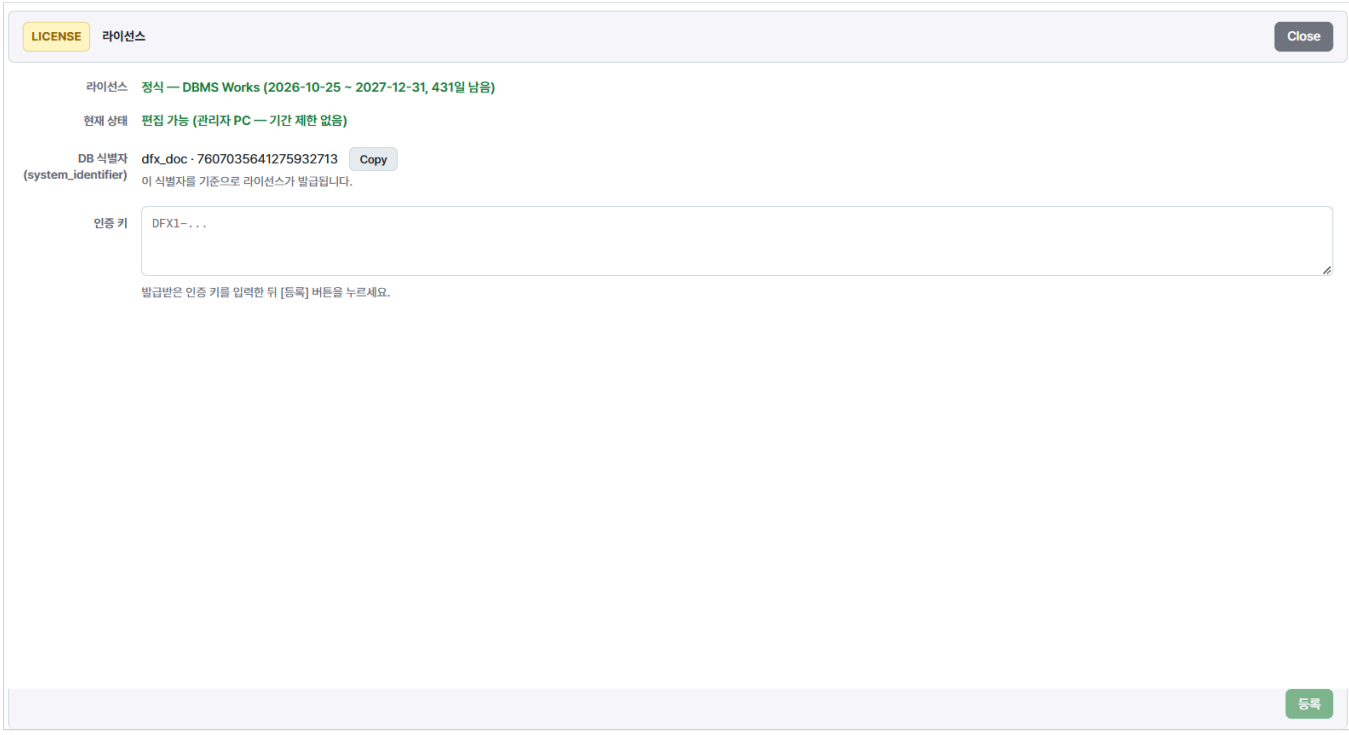


설치 전에 만들 데이터베이스 · 롤 · 스키마 · 오브젝트 · 접속 정보 · 라이선스를 한 번 더 보여 줍니다. 설치 중에는 단계만 표시합니다.



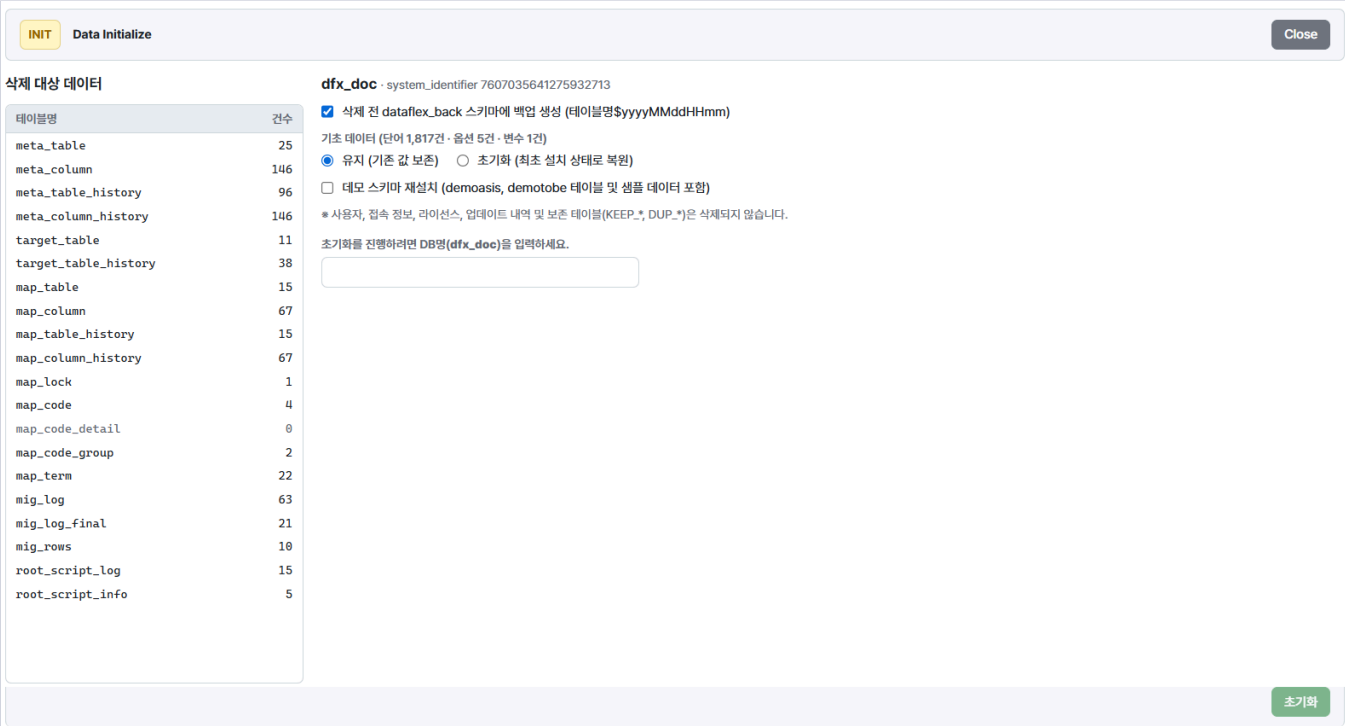
*완료 단계에서는 등록된 접속 정보, 설정 파일 위치, 라이선스 상태 및 초기 계정을 표시합니다. **DataFlex 시작**을 클릭하면 프로그램이 다시 실행됩니다.*

라이선스. 인증 키는 리포지토리 DB 서버의 system_identifier 에 묶어 발급하는 전자서명 키입니다. 프로그램은 내장된 공개키로 서명을 확인하므로, 키를 고치거나 다른 서버에 옮겨 쓸 수 없습니다. 인증 키를 넣지 않으면 설치한 날부터 7일간 무료 체험으로 동작하며, 기간이 지나거나 키가 맞지 않으면 조회 전용으로 실행됩니다. 끝나기 7일 전부터는 시작할 때 남은 날을 알립니다. 인증 키는 설치 때 넣거나 설치 후 [Admin] > [License] 화면에서 등록합니다(ADMIN 사용자).



License — 지금 라이선스와 적용 상태(편집 가능 · 조회 전용), 키 발급에 쓰는 DB 식별자(system_identifier)를 보여 줍니다. 인증 키를 등록하면 프로그램을 다시 시작하지 않아도 바로 적용됩니다(이미 열린 화면은 다시 열면 적용).

데이터 초기화. [Admin] > [Data Initialize] 는 리포지토리의 작업 데이터(메타 · 매핑 · 타겟 · 코드 매핑 · 용어 대응 · 이행 로그 · 실행 기록과 각 이력)를 비워 새 프로젝트를 시작할 수 있게 합니다. 삭제 전에 dataflex_back 스키마에 백업을 남길 수 있고, 기초 데이터는 유지하거나 최초 설치 상태로 복원하며, 데모 스키마를 다시 설치할 수 있습니다. 사용자 · 접속 정보 · 라이선스 · 업데이트 내역과 인덱스 · 제약 복구 기록(KEEP_* · DUP_*)은 삭제 대상에서 제외됩니다. 실수로 실행하지 않도록 DB명을 입력해야 진행됩니다.

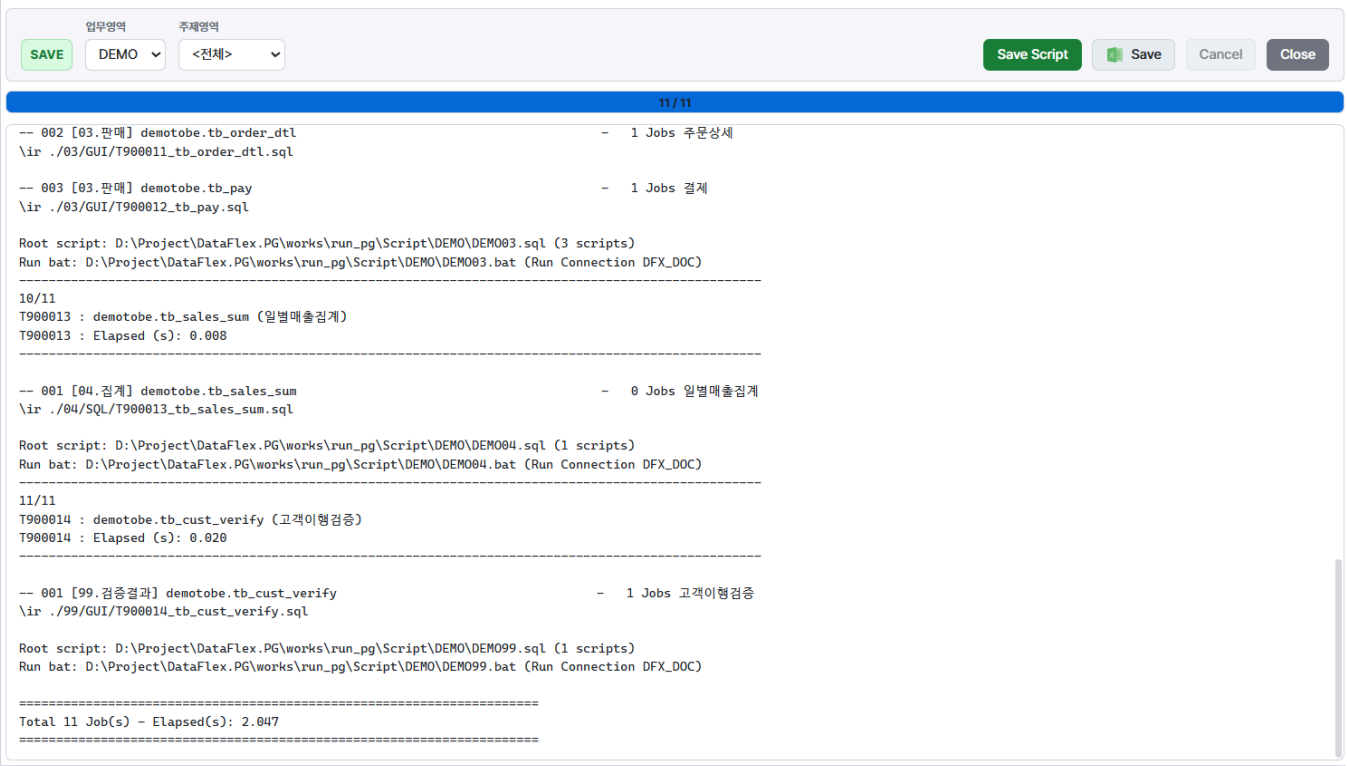


Data Initialize — 왼쪽은 삭제 대상 테이블과 건수, 오른쪽은 백업 · 기초 데이터 · 데모 스키마 옵션과 진행 단계입니다.

6. 산출물

산출물	형태	비고
매핑 정의	리포지토리 DB (엑셀로 저장 가능)	변경 이력 포함
이행 스크립트	PL/pgSQL 파일 — 테이블별 1개, 주제영역별 루트 스크립트 1개	**도구 없이 psql 로 실행 가능**
실행 순서	리포지토리 DB · 루트 스크립트	
실행 기록	리포지토리 DB 의 테이블	테이블 · 단계별 건수, 소요 시간, 오류

생성되는 스크립트는 **사람이 읽을 수 있는 일반 PL/pgSQL** 입니다. 실행 환경에 본 도구를 설치할 필요가 없고, DBA 가 검토 · 수정할 수 있으며, 프로젝트 종료 후에도 도구와 무관하게 자산으로 남습니다. 도구 종속에 따른 위험이 없습니다.



Save Script & Excel(Save As) — 매핑에서 스크립트를 만드는 화면입니다. 업무영역(DEMO) 전체를 한 번에: 대상 테이블 11개의 스크립트를 테이블마다 한 파일(`T900012_tb_pay.sql`)로, 주제영역마다 그것들을 차례로 부르는 루트 스크립트(`DEMO03.sql` — `\ir` 줄 하나가 테이블 하나)와 그것을 돌리는 실행 파일(`DEMO03.bat`)을 만듭니다. GUI 매핑은 `GUI\`, PL/pgSQL 방식 매핑은 `SQL\` 아래에 놓이고, 로그에 테이블별 소요 시간과 루트 스크립트의 구성이 남습니다(11개 테이블 2.0초). 같은 화면의 Save 는 매핑 정의서를 엑셀로 냅니다.

아래는 위 화면이 만든 파일 두 가지를 그대로 옮긴 것입니다 (샘플 DEMO, 2026-10-25 생성).

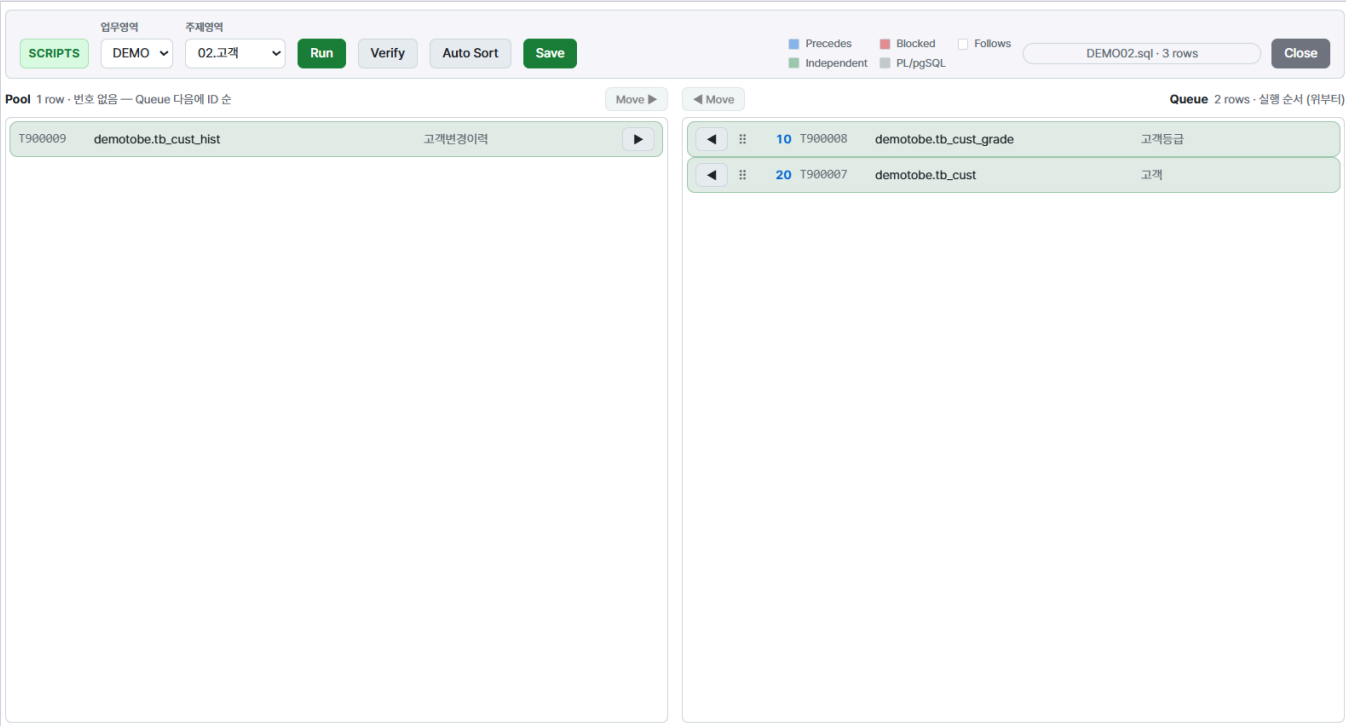
생성된 스크립트 — `03\GUI\T900010_tb_order.sql` 의 INSERT 부분. 대상 컬럼마다 데이터 타입 · NOT NULL · PK · 속성명이 주석으로 따라오고, SELECT 쪽에는 소스 컬럼의 타입(ASIS)이나 변환식이 적용된 대상 타입(TOBE)이 나란히 붙어 있어, 파일만 열어도 어느 컬럼이 어디서 어떻게 오는지 읽힙니다. Table Map 에서 병렬도를 2 이상으로 두어, 원본을 ctid 범위로 나눈 조각(`__DFX_FROM__` ~ `__DFX_TO__`)을 여러 세션이 이 문장으로 나눠 적재합니다. 앞뒤의 인덱스 제거 · 조각 나누기 · 실행 기록(p_joblog) · 예외 처리는 생략했습니다.

```
INSERT /* DataFlex.PG T900010 1 of 1 */
       INTO demotobe.tb_order
      (
          order_no                                                  -- character varying(20)  NN   Y     주문번호
        ,   order_dtm                                               -- timestamp(0) without time zone  NN   Y     주문일시
        ,   cust_id                                                 -- character varying(10)             고객ID
        ,   store_code                                              -- character varying(10)             매장코드
        ,   order_amt                                               -- numeric(15,2)               주문금액
        ------------------------------------------------------------------- 005
        ,   pay_code                                                -- character varying(20)             결제수단코드
        ,   status_code                                             -- character varying(20)             주문상태코드
        ,   created                                                 -- timestamp(0) without time zone             등록일시
      )
SELECT
          A1.ord_no AS order_no                                     -- ASIS: character varying(20)  NN   Y     주문번호
        , udf_char2date(A1.ord_dt || A1.ord_tm) AS order_dtm        -- TOBE: timestamp(0) without time zone  NN   Y     주문일시
        , A1.cust_no AS cust_id                                     -- ASIS: character varying(10)
        , A1.store_cd AS store_code                                 -- ASIS: character varying(10)
        , A1.ord_amt AS order_amt                                   -- ASIS: numeric(15,2)
        ------------------------------------------------------------------- 005
        , A1.pay_cd AS pay_code                                     -- ASIS: character varying(10)
        , A1.stat_cd AS status_code                                 -- ASIS: character varying(2)
        , localtimestamp(0) AS created                              -- TOBE: timestamp(0) without time zone             등록일시
FROM    (SELECT * FROM __DFX_SRC__ A1 WHERE ctid >= '__DFX_FROM__'::tid AND ctid < '__DFX_TO__'::tid)  A1         -- A900013                100,000
WHERE    1 = 1
```

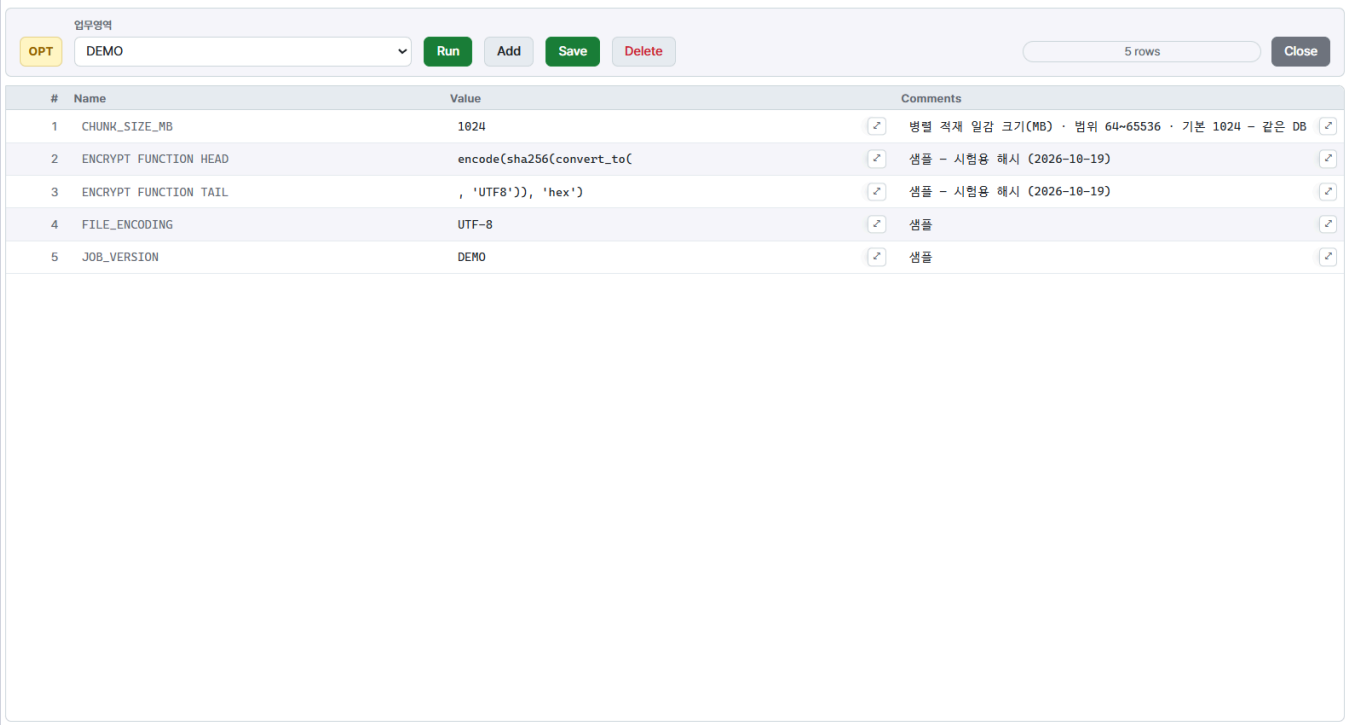
매핑 정의서 엑셀 — `02\T900007_demotobe.tb_cust.xlsx`. 시트 머리에 대상 테이블(demotobe.tb_cust · 고객)과 소스 테이블 세 개의 별칭 · 조인 방식 · 조인 조건(A1.cust_no = A2.cust_no …)이 오고, 그 아래가 컬럼 표입니다. 기존 프로젝트의 정의서 양식과 같은 꼴이라 그대로 산출물로 낼 수 있고, 매핑을 고치면 다시 저장하는 것뿐이라 화면과 정의서가 어긋나지 않습니다(아래는 열 몇 개를 줄여 옮긴 것).

No.	Column Name	속성명	Data Type	PK	NN	Source Table	Source Column	Source Type	매핑 로직	매핑 설명
1	cust_id	고객 ID	character varying(10)	Y	Y	cu_cust	cust_no	character varying(10)		
2	cust_name	고객명	character varying(50)		Y	cu_cust	cust_nm	character varying(50)		
3	rrn_enc	주민등록번호(암호화)	character varying(200)			cu_cust	rrn	character varying(13)		대상 컬럼이 암호화(ENCRYPT_YN) — 옵션 ENCRYPT FUNCTION HEAD · TAIL 로 감싸인다
4	tel	연락처	character varying(20)			cu_cust	tel	character varying(20)		
5	email	이메일	character varying(60)			cu_cust	email	character varying(100)	`left(A1.email, 60)`	원본 100 > 대상 60 — 잘라 넣는다
6	grade_code	등급코드	character varying(20)			cu_cust	grade_cd	character varying(10)	`coalesce((SELECT m.target_code FROM dataflex.map_code m WHERE … m.source_code = A1.grade_cd), A1.grade_cd)`	등급 코드 변환(MAP_CODE). 없는 코드 · NULL 은 그대로
7	join_date	가입일자	timestamp(0) without time zone			cu_cust	join_dt	character varying(8)	`udf_char2date(A1.join_dt)`	
8	status_code	상태코드	character varying(20)			cu_cust	stat_cd	character varying(2)		
9	zip	우편번호	character varying(6)			cu_cust_addr	zip	character varying(6)		대표 주소(rep_yn = Y) 인라인뷰
10	addr	주소	character varying(400)						`nullif(concat_ws(' ', A3.addr1, A3.addr2), '')`	기본 주소 + 상세 주소
11	birth_date	생년월일	timestamp(0) without time zone			cu_cust_dtl	birth_dt	character varying(8)	`udf_char2date(A2.birth_dt)`	
12	sex_code	성별코드	character(1)			cu_cust_dtl	sex_cd	character(1)		
13	job_code	직업코드	character varying(20)			cu_cust_dtl	job_cd	character varying(10)		

No.	Column Name	속성명	Data Type	PK	NN	Source Table	Source Column	Source Type	매핑 로직	매핑 설명
14	memo	메모	character varying(500)			cu_cust_dtl	memo	character varying(1000)	`left(A2.memo, 500)`	원본 1000 > 대상 500 — 잘라 넣는다
15	created	등록일시	timestamp(0) without time zone						`localtimestamp(0)`	업무영역 변수



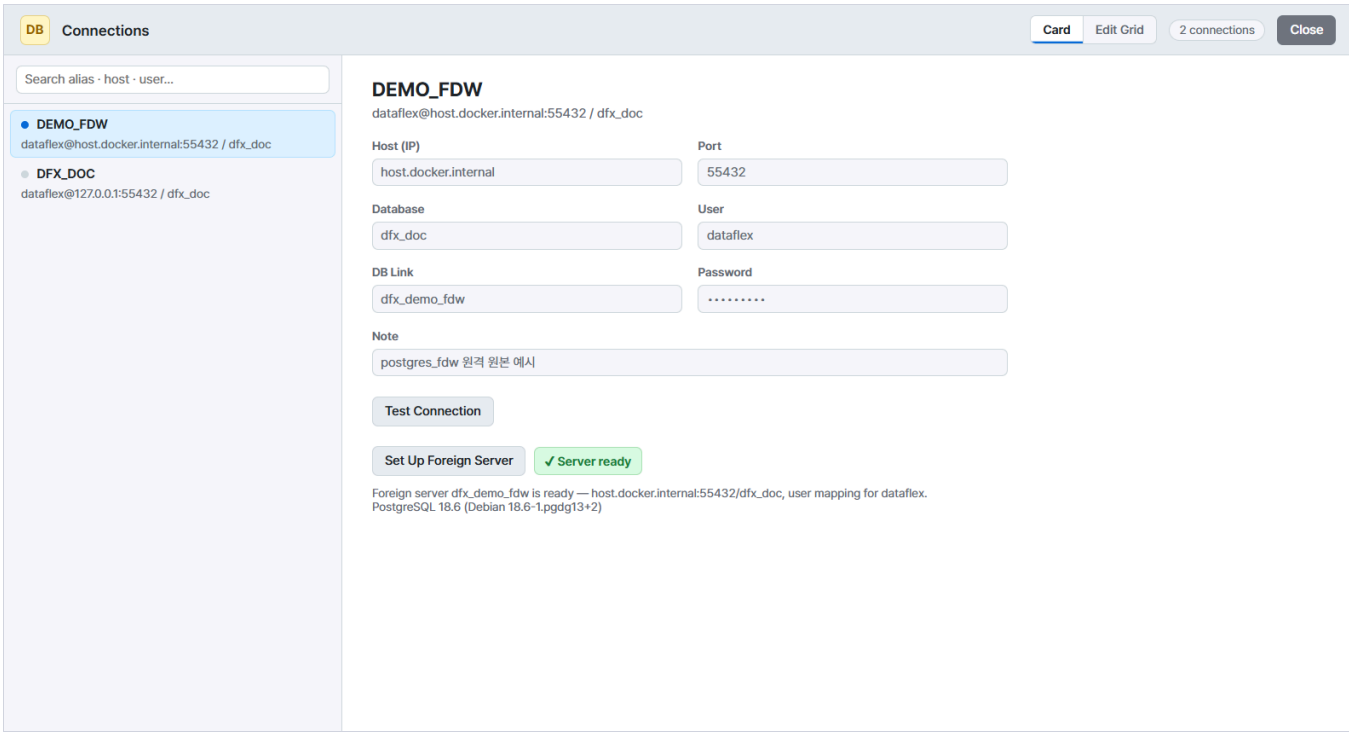
Ordering Scripts — 주제영역 하나(02.고객)의 루트 스크립트(DEMO02.sql) 안에서 테이블 스크립트가 불리는 차례를 정합니다. 대부분의 테이블은 아무 때나 돌아도 되므로 왼쪽 Pool 에 두고(번호 없음 — Queue 다음에 ID 순), 먼저 돌아야 할 것만 오른쪽 Queue 로 옮겨 위부터 번호를 받습니다(10 · 20). 어떤 테이블의 매핑이 다른 대상 테이블을 읽으면 그 테이블이 먼저 적재되어야 합니다 — 도구가 매핑에서 그 선후 관계를 읽어 줄마다 색으로 보이고(Precedes · Follows · 관계 없음 Independent · 어긋남 Blocked), Verify 는 지금 차례를 점검해 Blocked 를 알리며, Auto Sort 는 선후 관계에 맞게 자동으로 줄을 세웁니다. 어긋난 채 저장하면 빈 테이블을 읽게 되므로 저장 전에 한 번 더 묻습니다.



Option Values — 업무영역 단위의 스크립트 생성 옵션입니다. 병렬 조각의 일감 크기(CHUNK_SIZE_MB), 암호화 함수의 머리와 꼬리, 파일 인코딩, 작업 버전처럼 생성되는 모든 스크립트에 공통으로 들어가는 값을 한 곳에서 정합니다. 여러 줄 값은 전용 편집기에서 고칩니다.

7. 적용 절차

순서	내용	주체
1	리포지토리 DB 설치 — 설치 마법사(`DataFlexPG.exe -install`)로 데이터베이스 · 롤 · 권한 · 오브젝트 · 기초 데이터 · 접속 정보를 한 번에 설치 (5.7 참조)	DBA
2	원본 · 대상 접속 등록(다른 서버 원본은 외부 서버 준비), Meta Loader 로 테이블 정보 등록	이행 담당
3	매핑 — 담당자별 Table Map · Column Map 작성, Mapping Progress 로 진척 관리	이행 담당 · 업무 담당
4	리허설 — 스크립트 생성 · 실행 → 기록과 타임라인으로 지연 · 실패 테이블 식별 → 매핑 수정 → 재생성. 본 이행까지 반복	이행 담당 · DBA
5	본 이행 — 리허설에서 검증된 스크립트와 실행 순서를 그대로 사용	이행 담당 · DBA



Tool Connections — 도구가 여는 접속(원본 · 대상 · 메타 DB)의 목록입니다. 왼쪽에서 선택하면 오른쪽에 호스트 · 포트 · 데이터베이스 · 계정 · DB Link 가 보이고, Test Connection 으로 붙는지 확인합니다. DB Link 칸에는 그 원본을 읽을 외부 서버(postgres_fdw) 이름을 적으며, 외부 서버 · 사용자 매핑이 아직 없으면 Set Up Foreign Server 로 만듭니다. 비밀번호는 암호화되어 저장되며(그림에서는 가렸습니다) 고치는 것은 관리자만 할 수 있습니다. 적용 절차 2단계(접속 등록)의 화면입니다.

8. 적용 환경과 전제 조건

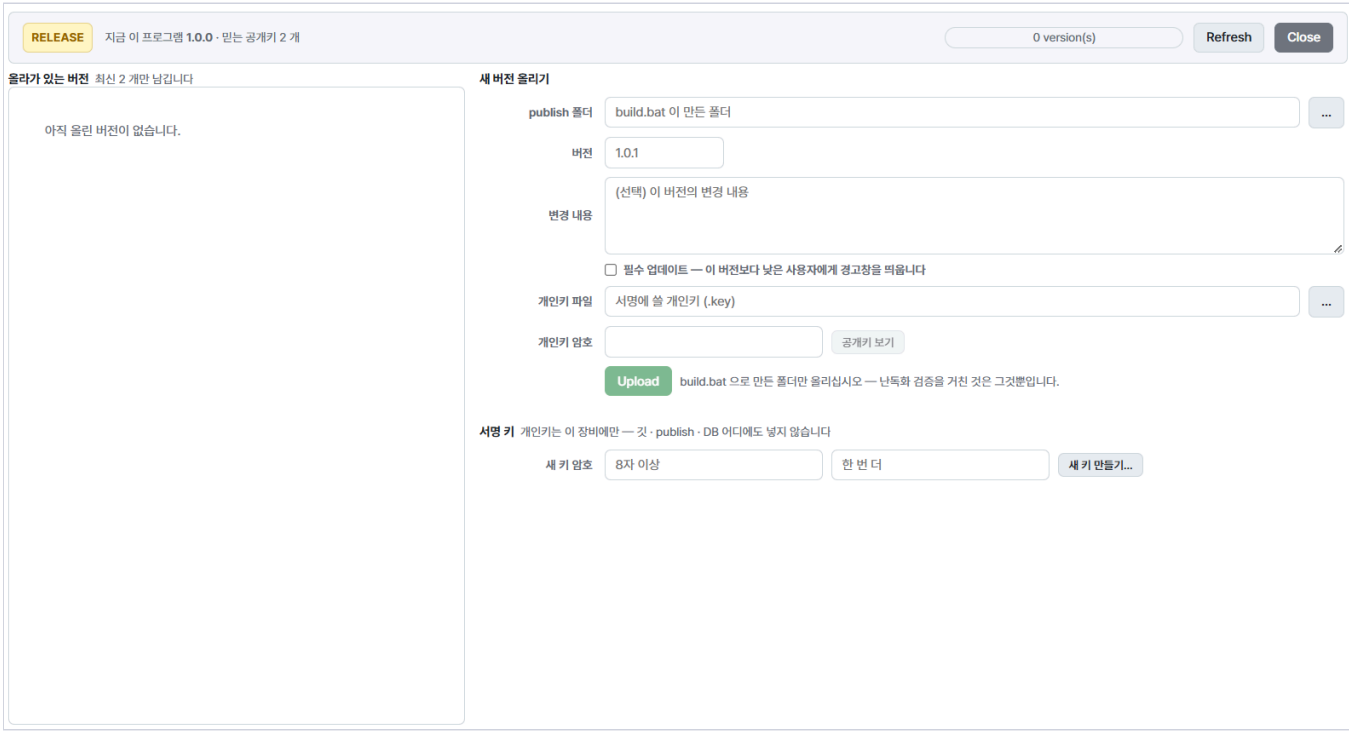
구분	내용
클라이언트	Windows 10 이상. 실행 파일 1개 + 폴더 몇 개(psql · lang 등), .NET 설치 불필요. Microsoft Edge WebView2 런타임 필요 (Windows 11 기본 포함)
리포지토리 DB	PostgreSQL 14 이상(MERGE 적재는 15 이상). 설치 마법사가 dataflex 롤과 스키마 3개(dataflex · dataflex_dup · dataflex_back), 확장 dblink · postgres_fdw 를 만듭니다
확인한 DB	PostgreSQL 16.15 · 18.6 — 설치 마법사로 새로 설치한 리포지토리 DB 에서 이행 실행 · 대사 · 라이선스 · 데이터 초기화까지 전 과정을 자동 시험(475개 항목)과 실제 앱 조작으로 확인했습니다
스크립트 실행	psql (프로그램과 함께 배포)

전제 조건 — 리포지토리 DB 롤의 권한. dataflex 롤이 원본을 읽고 대상 스키마에 적재 · DDL 을 수행하므로, 설치 마법사가 다음 권한을 줍니다: 조회 · 적재(`pg_read_all_data` · `pg_write_all_data`), 진행 확인 · 세션 정리(`pg_read_all_stats` · `pg_signal_backend`), 외부 서버(`postgres_fdw` 사용), 라이선스 확인(`pg_control_system()` 실행). 대상 테이블의 인덱스 · 제약 · 트리거를 떼고 붙이려면 그 테이블의 주인 롤 권한이 필요합니다(샘플은 `GRANT demotobe TO dataflex`). 전체 목록은 설치 스크립트에 명시되어 있으며, **이행 기간에 한해 부여하고 종료 후 회수**하는 운영을 권장합니다.

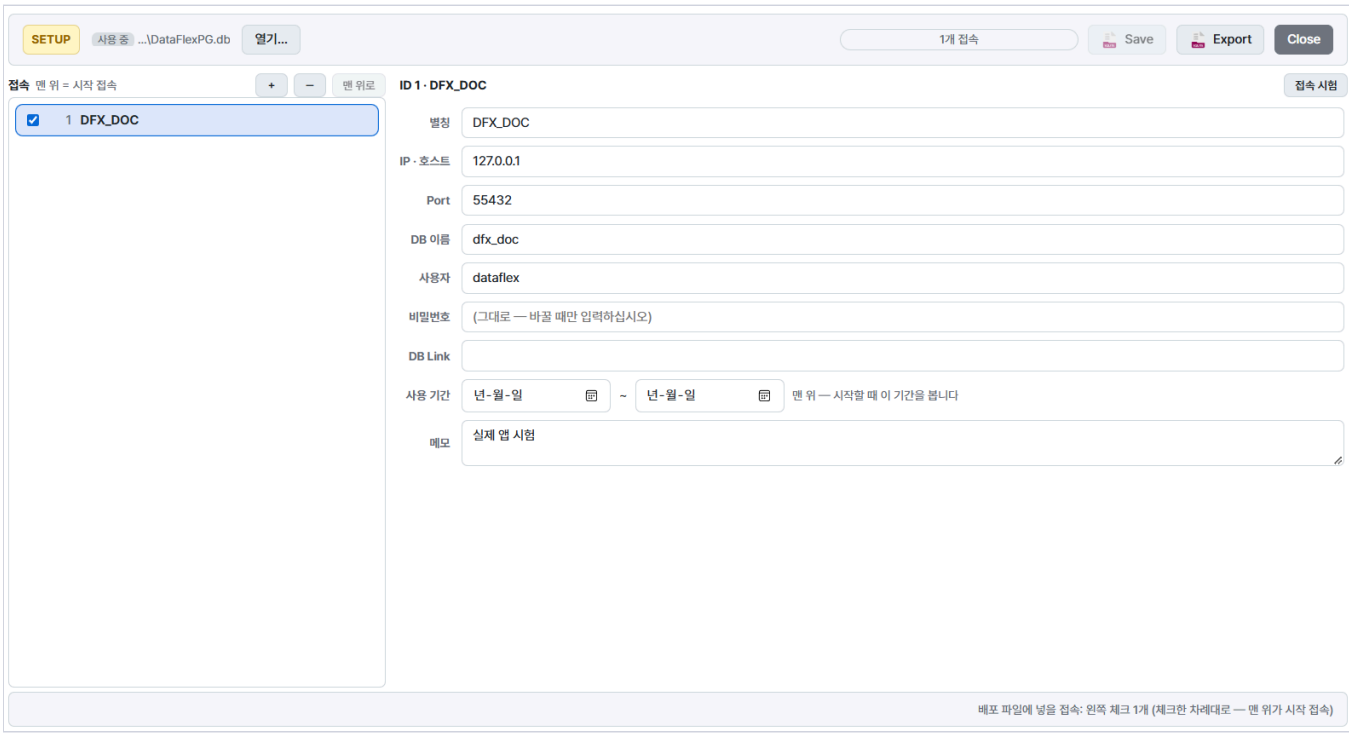
9. 보안

- 데이터는 DB 안에서 이동하며 클라이언트 PC 를 거치지 않습니다.
- 접속 비밀번호는 암호화하여 보관합니다.
- 사용자 구분: 관리자 / 일반 / 읽기 전용(GUEST). 읽기 전용은 화면과 처리 단계 양쪽에서 차단되며, 상세는 5.6 참조.
- 개인정보 컬럼은 매핑 단계에서 암호화 함수를 적용해 이행할 수 있습니다 (암호화 함수는 고객사 표준을 지정).
- 매핑 · 메타의 변경은 변경자 · 시각과 함께 이력으로 남습니다.
- 인증 키는 리포지토리 DB 서버의 system_identifier 에 묶인 전자서명 키입니다. 프로그램에는 확인용 공개키만 들어 있고, 키를 만드는 개인키는 들어 있지 않습니다.
- **프로그램 업데이트는 리포지토리 DB 로 배포합니다.** 별도 배포 서버 · 파일 공유 · 외부 인터넷 접속이 필요 없습니다. 새 버전은 전자서명되어 올라가며, 클라이언트는 프로그램에 내장된 공개키로 **서명을 확인한 패키지만** 받습니다 — 리포지토리 DB 에 접근할 수 있

는 사람이 패키지를 바꿔 넣어도 설치되지 않습니다. 받기는 사용 중에 백그라운드로, 교체는 다음 실행 때 이루어지며 실패하면 이전 버전으로 그대로 실행됩니다. 누가 어느 버전을 받았고 적용했는지는 이력으로 남습니다.



Publish Release — 새 버전을 배포하는 관리자 화면입니다(등록된 장비에서만 열립니다). 왼쪽은 리포지토리 DB 에 올라가 있는 버전으로, 서명이 프로그램에 내장된 공개키로 확인되는지 · 필수 업데이트인지 · 받음 · 적용 · 실패 수가 보입니다. 오른쪽에서 build.bat 이 만든 publish 폴더를 선택하고 개인키 파일과 암호로 서명해 올립니다. 문제 있는 버전은 배포 중지로 거두고, 옛 버전과 맞지 않는 변경을 낼 때는 필수 업데이트를 체크해 옛 버전 사용자에게 경고창을 띄웁니다.



*Deploy Setup은 배포용 설정 파일(`DataFlexPG.db`)을 생성하는 관리자 화면입니다. 등록된 장비에서만 열 수 있습니다. 접속 정보와 사용 기간을 입력한 뒤 배포할 접속 항목을 선택하고 **Export**를 클릭해 배포용 파일에 저장합니다. 목록의 맨 위에 있는 접속 정보가 프로그램 시작 시 사용됩니다.*

10. 검증용 샘플

레거시 14개 테이블을 대상 11개 테이블로 이행하는 샘플 프로젝트를 제공합니다. 문자열 날짜(자정을 `240000` 으로 기록한 행 포함), PK 없는 중복 데이터, 변환 테이블에 없는 코드, 대상보다 긴 값 등 **실제 레거시에서 발생하는 사례를 의도적으로 포함**했으며, 조인 · MERGE · 병

렬 조각 적재 · 중복 제거 · 코드 변환 · 암호화 · SQL 방식 매핑 · IDENTITY 컬럼 이행을 도입 전에 직접 확인할 수 있습니다 (주문 10만 건 규모, 수 초 소요). 샘플은 리포지토리 DB 안의 두 스키마(demoasis · demotobe)에 들어가며, 지금 상태(테이블 · 데이터 · 매핑)는 DDL · COPY 스크립트로 뽑아 두어 어느 서버에서나 같은 상태를 다시 만들 수 있습니다. 설치 마법사에서 데모 스키마를 함께 설치할 수 있고, Data Initialize 의 데모 스키마 재설치로 처음 상태로 되돌릴 수 있습니다.

11. 제약과 향후 계획

- 원본 · 대상 모두 **PostgreSQL** 을 대상으로 합니다.
- 클라이언트는 Windows 전용입니다.
- 설치 마법사는 PostgreSQL 14 이상을 대상으로 합니다. MERGE 적재는 PostgreSQL 15 이상에서 돌고, MERGE 의 새로 넣은 · 바뀐 건수는 17 이상에서만 따로 기록합니다.
- 건수 대사(Rows Compare)의 예상 건수는 GUI 매핑만 셉니다. PL/pgSQL 로 직접 쓴 SQL 방식 매핑은 읽는 문장을 알 수 없어 적재 건수만 보입니다.